15장
프로그램 화면 구성하기: tkinter

마지막 장에서는 프로그램의 화면을 구성합니다. `tkinter`라는 패키지를 활용하는데, 윈도우 환경에서 `tkinter` 화면은 저마다 구성된 것에 상당 부분을 화면을 수 있습니다. 주제에 맞는 부분을 선별해 서비스 사용하지만, 우분투 같은 리눅스 플랫폼에서 좋아 가이드를 제대로 구성할 수 없습니다. 때문에 하나의 플랫폼에서 프로그램을 작성했더라도 이 장에서는 주피터 노트북을 사용해 실행합니다.

15.1

tkinter 살펴보기

앱, 게임 등 대부분 프로그램은 사용자가 화면을 클릭하거나 터치해서 동작을 실행하는, 사용하기 편리하고 직관적인 화면으로 되어 있습니다. 그동안 파이썬 프로그래밍의 기본적인 요소를 배우고 프로젝트도 진행했으니 이제는 화면을 만들어 프로그램의 완성도를 높여 봅시다.

15.1.1 GUI란

앞에서 작성한 프로그램은 우리가 평소 사용하던 프로그램 모습과는 조금 달랐죠? 실제 사용하는 프로그램은 코드가 보이지 않고 텍스트나 이미지가 나오는 화면과 버튼 등이 주로 보이죠. 그래서 버튼을 클릭하거나 화면을 터치해 프로그램을 조작할 수 있습니다.

이는 프로그램에 UI를 입혔기 때문입니다. UI란 User Interface의 약자로 사용자 인터페이스를 의미합니다. 사용자 인터페이스란 사용자와 코드 사이를 연결하는 모든 것을 의미합니다. 사용자 화면, 안내 음성 등이 이에 해당됩니다. UI 앞에 graphical의 약자인 G를 붙여 GUI라고 합니다. GUI(Graphical User Interface)는 프로그램의 기능을 다양한 그래픽으로 나타낸 것을 의미합니다. 사용자와 프로그램이 소통할 수 있게 도와주는 UI 중에서도 시각적 부분이 GUI입니다.

처음 프로그래밍을 배우면서 '내가 생각한 프로그램이 아닌데?'라고 느꼈다면 이런 사용자 화면(GUI)이 없어서 그랬을 겁니다. 지금까지 작성한 프로그램의 형태는 사용자가 직접 프로그램에 입력값을 넣고 실행결과를 결과창에서 확인하는 형태였습니다. 그러나 사용자 화면이 있는 프로그램에서는 사용자가 코드를 볼 수 없고, 사용자 화면만 볼 수 있습니다. 사용자 화면에 값을 입력하고 버튼을 클릭하는 등의 동작을 하면 이에 맞는 결괏값이 결과창이 아닌 사용자 화면에 나타납니다. 사용자 화면이 없는 경우와 있는 경우를 비교하면 다음과 같습니다.



▲ 그림 15-1 프로그램 형태

파이썬에는 이런 GUI 프로그래밍을 돕는 다양한 라이브러리가 있습니다. 그중에서 파이썬 표준 라이브러리에 속한 `tkinter`를 다루겠습니다.

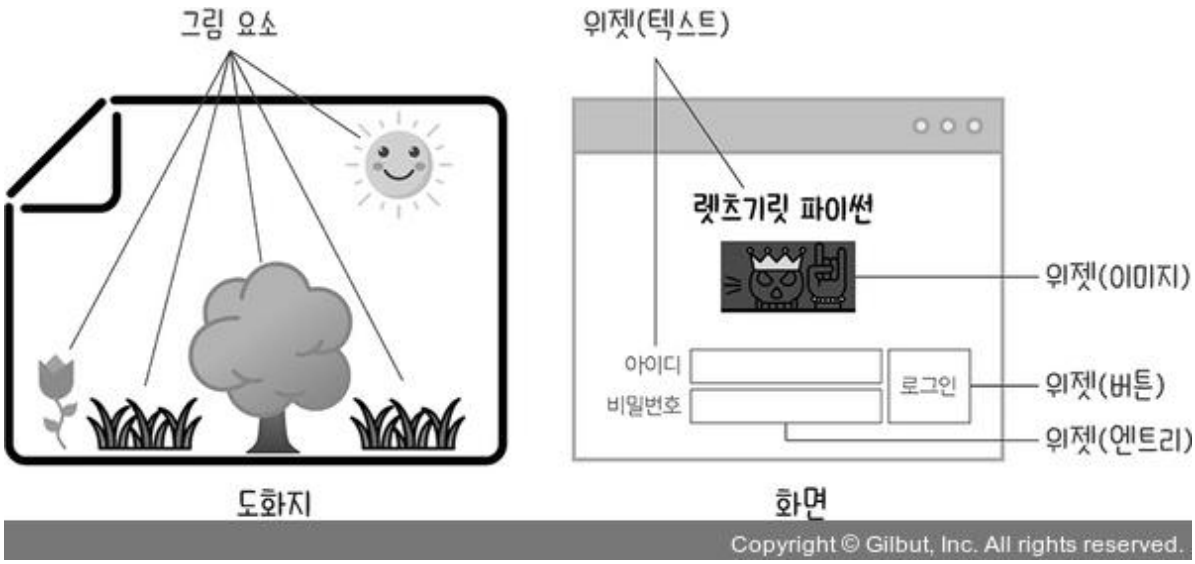
15.1.2 파이썬의 표준 GUI tkinter

파이썬은 범용성이 뛰어난 프로그래밍 언어인 만큼 적용할 수 있는 GUI가 다양합니다. PyQt처럼 정교한 GUI를 만들 수 있는 도구도 있지만, 이 책에서는 가장 기본 도구이 `tkinter`를 간단하게 배워 봅니다. `tkinter` 자체도 기능이 매우 많아서 GUI의 핵심이 되는 레이블, 버튼, 엔트리, 메시지박스, 콤보박스 위주로 살펴보겠습니다. 더 많은 기능은 다음 사이트를 참고하세요.

- python.org의 공식 tkinter 문서 https://docs.python.org/ko/3/library/tkinter.html?highlight=tkinter#module-tkinter
- TkDocs Tutorial https://tkdocs.com/tutorial/index.html

GUI를 그림 그리기에 비유하면 이해하기 쉽습니다. 그림을 그리려면 가장 먼저 도화지나 연습장 등이 필요하죠. 도화지를 준비하는 것처럼 GUI를 구성하기 전에 바탕이 되는 화면을 만들어야 합니다. `tkinter`에서는 이를 라이브러리 내부에 있는 `Tk` 클래스의 객체를 생성해 만듭니다. 14장에서 클래스를 정의하면 객체를 만들 수 있었죠? 화면을 만들 수 있는 클래스가 이미 `tkinter`에 정의되어 있고, 우리는 이 클래스를 바탕으로 객체만 만들면 됩니다.

그림을 그릴 때는 도화지 위에 원하는 구도에 맞게 꽃, 나무, 풀 등 그리려는 대상을 하나씩 그립니다. 프로그래밍에서도 GUI에 필요한 요소들을 화면에 배치하는데, 이런 요소들을 위젯이라고 합니다. 위젯에는 텍스트, 이미지, 버튼 등 다양한 종류가 있습니다. 위젯 역시 `tkinter`에 이미 클래스로 정의되어 있습니다. 따라서 위젯을 나타내는 클래스의 객체를 생성한 다음, `Tk` 클래스의 객체 위에 배치하면 됩니다.



▲ 그림 15-2 도화지(화면)에 요소(위젯) 배치하기

15.1.3 tkinter 사용하기

tkinter 추가하기

GUI와 `tkinter`의 관계를 알아봤으니 사용법을 배워 봅시다. 먼저 `tkinter`의 정보를 `help()`로 알아보죠.

```
import tkinter

help(tkinter)
```

```
실행결과
Help on package tkinter:

NAME
    tkinter - Wrapper functions for Tcl/Tk.
…
```

실행결과를 보니 `tkinter`는 패키지입니다. 패키지는 내부에 여러 모듈이 있다고 했습니다. 패키지에 있는 모듈은 다음 형식으로 사용합니다. 사용법은 보이는 것처럼 `from 큰범위 import 작은범위`입니다.

형식

```
from 패키지 import 모듈                # 패키지 중 특정 모듈만 사용하고 싶을 때
from 패키지.서브패키지 import 모듈     # 패키지에 서브패키지가 있을 때 1
from 패키지 import 서브패키지          # 패키지에 서브패키지가 있을 때 2
```

`tkinter`를 다룰 때는 `from tkinter import *`로 사용하겠습니다. `*`는 모든 것을 포함한다는 의미입니다. 그렇다면 `import tkinter`와 다른 점은 무엇일까요?

```
import tkinter

root = Tk()              # 오류 발생
# root = tkinter.Tk()    # 오류 발생 안 함
```

```
실행결과
NameError: name 'Tk' is not defined
```

`Tk`는 `tkinter` 패키지에 속한 클래스입니다. 이때 `import tkinter`로만 작성하면 클래스명 앞에 항상 `tkinter`를 추가해야 합니다. `root = Tk()`를 주석 처리하고 마지막 줄의 주석 처리를 해제한 후 실행하면 오류가 발생하지 않습니다.

```
from tkinter import *

root = Tk()
```

앞에 `tkinter`를 넣지 않아도 오류가 발생하지 않습니다. 그래서 `tkinter` 패키지의 명령어를 사용할 때 일일이 `tkinter`를 추가하지 않게 `from tkinter import *`를 사용합니다.

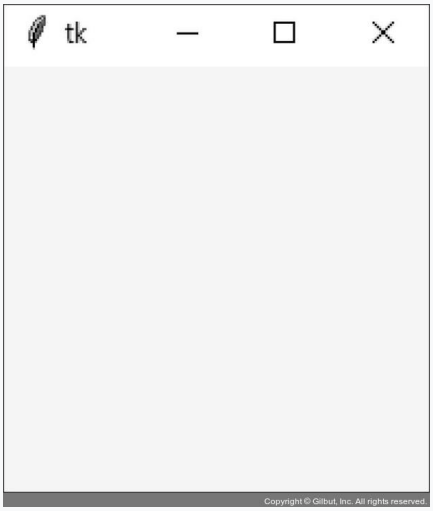
tkinter 창 만들어 띄우기

앞에서 작성한 `root = Tk()`는 `Tk` 클래스의 객체를 만드는 코드입니다. `Tk`는 창을 생성하는 클래스입니다. 따라서 `Tk` 클래스로 만든 객체 `root`가 생성됩니다. 그런데 객체를 생성한 후 아무 일도 일어나지 않습니다. `Tk` 클래스에는 결과를 화면으로 보여 주는 메서드가 따로 있습니다. 다음처럼 프로그램 마지막에 `root.mainloop()`를 추가해 코드를 실행해 봅시다.

```
from tkinter import *

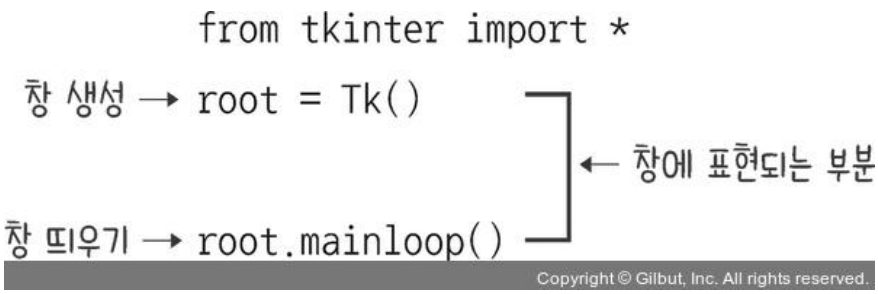
root = Tk()
root.mainloop()
```

실행결과



프로그램을 실행하는 브라우저 위로 별도의 창(윈도우)이 하나 뜹니다. `root` 객체가 실행한 `mainloop()`는 프로그램의 메인 창 `root`를 화면에 띄우는 `Tk` 클래스의 메서드입니다. 프로그램의 바탕이 되는 화면을 만들기 때문에 객체명을 `root`(뿌리, 근원)로 작성합니다. 그림 그리는 데 필요한 도화지를 만든 셈입니다.

`root` 객체가 생성된 직후부터 `root.mainloop()` 바로 위까지의 코드를 실행한 결과가 창에 나타납니다. 따라서 창에 표현될 내용은 두 줄 사이에 들어가야 합니다.

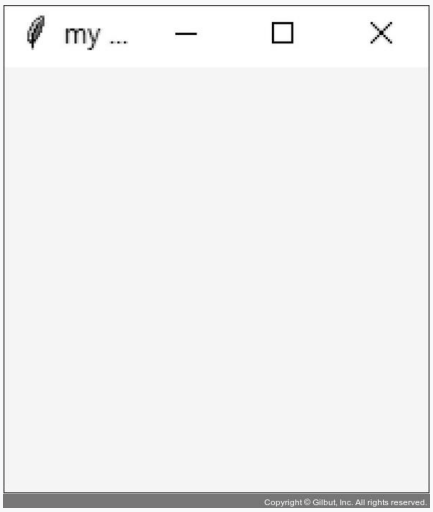


그러므로 창이 띄워져 있으면 프로그램이 실행 중인 상태입니다. 창이 떠 있는 상태에서 주피터 노트북의 코드 입력창 왼쪽을 보면 [`*`] 표시가 있습니다. 이는 프로그램이 '실행 중'이라는 의미입니다. 창 오른쪽 위에 있는 X 표시를 눌러 창을 닫으면 프로그램 실행이 멈추고 [`*`] 부분이 코드 줄 수를 나타내는 숫자로 바뀝니다.

창 상단에 창 이름이 나타납니다. 별도로 창 이름을 설정하지 않으면 기본값으로 tk라고 뜹니다. 창 이름은 바꿀 수 있습니다. 그래프에 제목을 붙일 때처럼 창 이름도 `title()`로 설정합니다. 원하는 이름을 `title()`의 소괄호 안에 넣고, `root = Tk()`와 `root.mainloop()` 사이에 추가합니다.

```
root = Tk()
root.title('my first tkinter')
root.mainloop()
```

실 행 결 과



상단 바에 있는 창 이름이 ‘my first tkinter’로 바뀝니다. 창의 가로 길이가 짧아 제목이 잘려 보이죠? 창의 크기를 늘려 봅시다. 마우스로 늘려도 되지만, 코드로 조절할 수도 있습니다. 창의 크기는 `geometry()`로 설정합니다. 역시 소괄호 안에 창의 크기를 ‘너비x높이’ 형태로 넣습니다. 이때 x는 알파벳 소문자고 앞뒤로 띄어 써서는 안 됩니다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200')
root.mainloop()
```

실 행 결 과



처음 창을 생성했을 때보다 창의 너비가 더 넓어졌습니다. 여기서 숫자 400과 200은 픽셀을 의미합니다. 앞의 코드는 너비 400픽셀, 높이 200픽셀인 창을 만들라는 뜻이죠.

창의 크기뿐만 아니라 창이 나타나는 위치도 설정할 수 있습니다. 앞서 작성한 창의 크기 설정 뒤에 다음과 같이 가로 시작 위치와 세로 시작 위치를 추가하면 됩니다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200+0+300')
root.mainloop()
```

코드를 실행하면 창이 모니터 화면의 왼쪽 끝 중간쯤에 나타납니다(나타나는 위치는 모니터 크기에 따라 조금씩 다를 수 있습니다). 가로가 0이므로 가장 왼쪽에, 세로가 300이므로 위에서 300번째 픽셀에 해당하는 위치에 창이 타납니다. 숫자를 바꾸며 다양하게 실습해 보세요.

창의 크기를 변경하지 못하게 할 수도 있습니다. `resizable()` 명령어로 창의 너비와 높이를 변경할 수 있게 할지 설정합니다. 변경할 수 있다면 `True`를, 변경할 수 없다면 `False`를 넣습니다. 예를 들어, 다음과 같이 작성하면 창의 너비는 변경할 수 없지만, 높이는 변경할 수 있습니다.

```
root.resizable(False, True)
```

1분 퀴즈

1 다음 설명 중 틀린 것을 고르세요.

① `tkinter`는 파이썬에서 화면을 표현하는 GUI 라이브러리다.

② `root = Tk()`는 `Tk` 클래스의 객체를 만드는 코드다.

③ 화면에 표현할 코드는 `root.mainloop()` 뒤에 작성한다.

④ `root.geometry()`로 창의 크기를 조절할 수 있다.

⑤ `root.resizable()`로 창의 크기를 변경할 수 없게 설정한다.

정답 및 해설

15.2

위젯 다루기

지금까지 프로그램의 바탕이 되는 사용자 화면을 만들어 보았습니다. 이제 창에 배치할 위젯을 몇 가지 배워 봅시다. 위젯에는 텍스트, 이미지, 버튼, 콤보박스, 메시지박스 등 여러 종류가 있습니다. 이런 위젯을 화면에 표시하려면 다음 두 가지 작업을 해야 합니다.

1 사용하려는 위젯의 객체 만들기

2 만든 위젯을 창에 배치하기

이 두 가지를 모두 진행해야 위젯을 창에 표시할 수 있습니다.

15.2.1 레이블

첫 번째 위젯으로 레이블(Label)을 다뤄 보겠습니다. 레이블로는 텍스트와 이미지를 표현할 수 있습니다.

> TIP Label은 레이블 또는 라벨로 읽습니다. 다른 곳에서 `Label` 위젯을 라벨로 표현해도 같은 것으로 보면 됩니다.

`help()`로 레이블에 대한 정보를 간략히 살펴봅시다.

```
from tkinter import *

help(Label)
```

실행결과

```
Help on class Label in module tkinter:

class Label(Widget)
    …
| Methods defined here:
|
| __init__(self, master=None, cnf={}, **kw)
|     Construct a label widget with the parent MASTER.
|
|     STANDARD OPTIONS
|
|         activebackground, activeforeground, anchor,
|         background, bitmap, borderwidth, cursor,
|         disabledforeground, font, foreground,
|         highlightbackground, highlightcolor,
|         highlightthickness, image, justify,
|         padx, pady, relief, takefocus, text,
|         textvariable, underline, wraplength
|
|     WIDGET-SPECIFIC OPTIONS
|
|         height, state, width
|
| ----------------------------------
…
```

`Label`은 많은 요소를 포함하고 있는 클래스입니다. 생성자 `__init__` 부분을 봅시다. 생성자 내부에 `STANDARD OPTIONS`가 있습니다. 이 중에서 주로 사용할 옵션만 굵게 표시했습니다. 모든 것을 다 알 필요는 없으니 부담 갖지 말고 훑어만 보세요. 차차 설명하겠습니다.

텍스트 레이블 만들기
텍스트 레이블을 만들어 봅시다. 먼저 `Label` 클래스의 객체를 만들어야 합니다. 레이블 객체는 `Label()`로 만들고, 텍스트는 `text` 옵션으로 나타냅니다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200+0+300')

label1 = Label(root, text='안녕하세요!')

root.mainloop()
```

실행결과



`Label` 클래스의 객체 `label1`을 생성합니다. 위젯의 객체를 만들 때는 첫 인자로 위젯을 띄울 창을 정해야 합니다. 여기서는 `root` 객체를 넣습니다. 그리고 창에 나타날 텍스트를 `text` 옵션으로 추가합니다.

그러나 창에 텍스트가 나타나지 않죠? 왜 그럴까요? 여기도 역시 레이블 객체를 생성하기만 했을 뿐 창에 배치하고 보여 주는 명령어가 없기 때문입니다. 위젯을 창에 표시하려면 `pack()` 메서드를 호출해야 합니다(위젯을 만들기만 해서는 화면에 나타낼 수 없습니다. 위젯의 위치를 지정해야 화면에 나타나는데, `pack()`은 위치를 지정하는 방법 중 하나입니다. `pack()`에 관한 설명은 뒤에 나옵니다). `label1.pack()`을 추가해 다시 실행해 봅시다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200+0+300')

label1 = Label(root, text='안녕하세요!')
label1.pack()

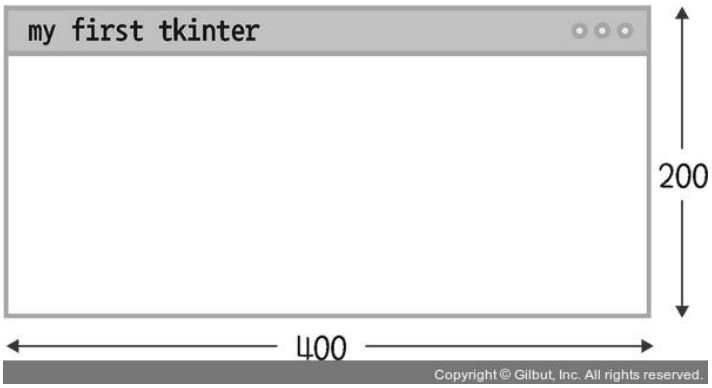
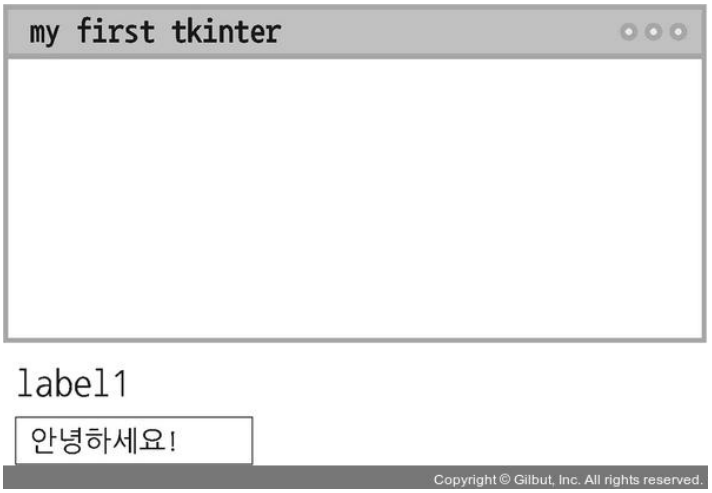
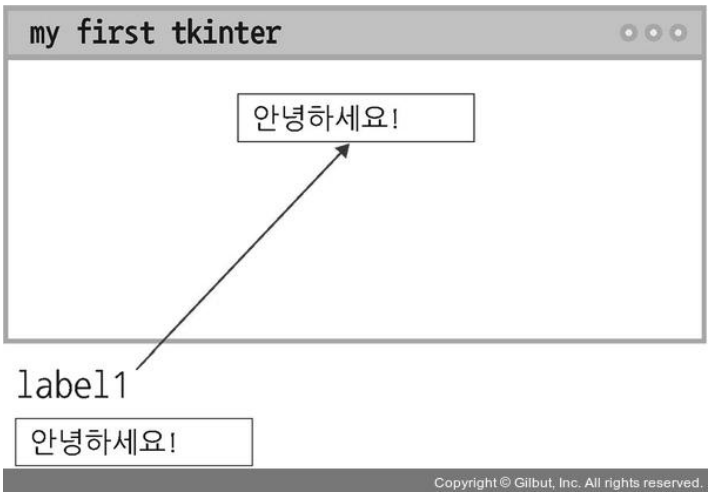
root.mainloop()
```

실행결과



이번에는 창에 텍스트가 나타납니다. 코드의 실행 과정은 다음과 같습니다.

▼ 표 15-1 코드 실행 과정

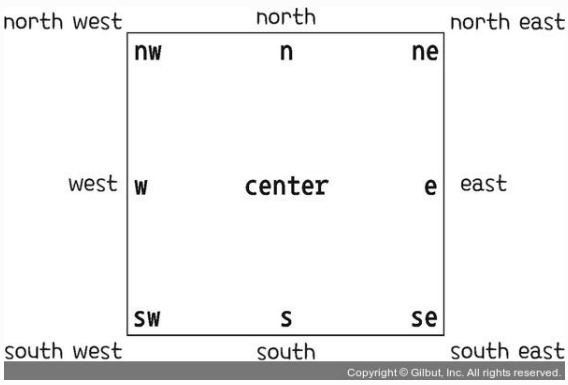
실행 순서	실행되는 명령어	컴퓨터에서 일어나는 일
1	`root = Tk()`	창 생성하기  Copyright © Gilbut, Inc. All rights reserved.
2	`root.title('my first tkinter')`	
3	`root.geometry('400x200')`	
4	`label1 = Label(root, text='안녕하세요!')`	`label1` 객체 생성하기  Copyright © Gilbut, Inc. All rights reserved.
5	`label1.pack()`	`label1` 객체의 위치 지정하기  Copyright © Gilbut, Inc. All rights reserved.
6	`root.mainloop()`	생성한 창 띄우기


my first tkinter
안녕하세요!
Copyright © Gilbut, Inc. All rights reserved.

NOTE 텍스트 레이블에 사용 가능한 옵션

레이블로 텍스트를 나타낼 때 글꼴, 글씨 크기, 색상, 테두리 모양 등 다양한 옵션을 추가할 수 있습니다. 주요 옵션은 다음과 같습니다.

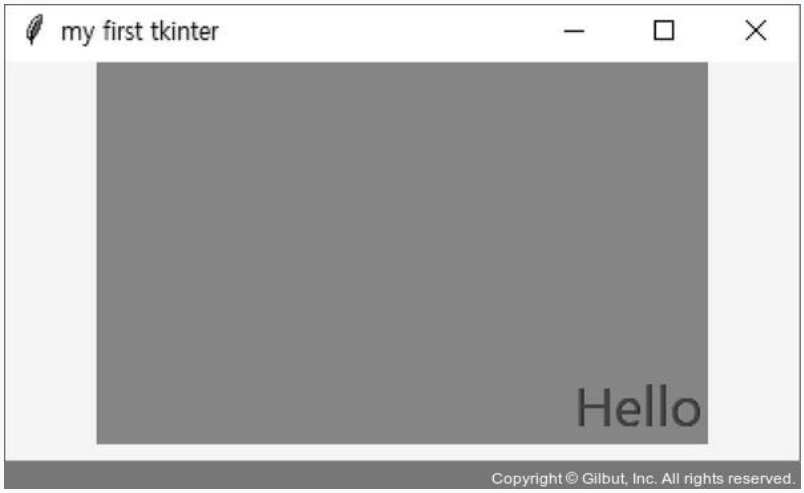
▼ 표 15-2 주요 옵션

옵션	설명
`font=('글꼴', 크기)`	글꼴과 크기 설정
`fg='색상'`	텍스트 색상 설정
`bg='색상'`	텍스트 바탕색 설정
`relief=테두리모양`	레이블 테두리 모양 설정 `flat`(테두리 없음, 기본값), `solid`(굵은 테두리), `groove`(움푹 파인 테두리), `ridge`(돌출된 테두리), `raised`(레이블 전체가 돌출), `sunken`(레이블 전체가 파임)
`borderwidth=테두리두께`	레이블 테두리 두께 설정
`width=너비`	레이블 너비 설정
`height=높이`	레이블 높이 설정
`anchor=정렬`	레이블 내부 정렬 방법 설정(`center`, `e`, `w`, `s`, `n`, `se`, `sw`, `ne`, `nw`) 

다음처럼 레이블의 객체를 생성할 때 필요한 옵션을 추가합니다.

```
label1 = Label(root, text='Hello', font=('Malgun Gothic', 20), fg='blue', bg='gray', wi
dth= 20, height=5, anchor='se')
```

실 행 결 과



더 많은 옵션은 https://tkdocs.com/shipman/label.html 문서를 참고하세요.

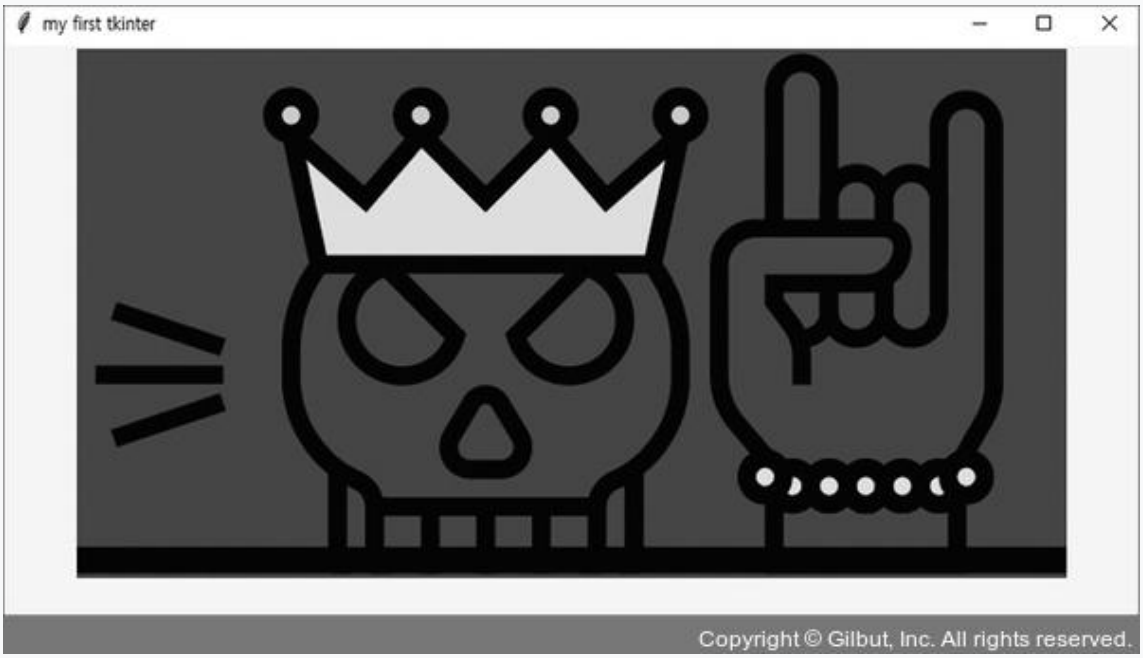
이미지 레이블 만들기

이미지 레이블도 만들 수 있습니다. 텍스트 레이블과 다르게 이미지 레이블은 레이블 생성 전에 이미지 객체인 `PhotoImage` 클래스의 객체를 만들어야 합니다. `PhotoImage`의 객체를 생성한 후, 레이블 객체를 생성할 때 `image` 옵션으로 나타낼 `PhotoImage` 객체를 추가합니다. 이미지 크기에 맞게 `tkinter` 창의 크기도 800x400으로 늘려 봅시다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('800x400')

letsgetit = PhotoImage(file='lets get it.png')  # PhotoImage 객체 생성
img_label = Label(root, image=letsgetit)        # 레이블 객체 생성
img_label.pack()
root.mainloop()
```

실행결과



창에 표시할 이미지 파일 하나를 가져와 현재 작업 중인 파이썬 파일과 같은 폴더에 넣습니다(제공하는 lets get it.png 파일을 사용해도 됩니다). `PhotoImage` 클래스로 이미지 객체 `letsgetit`을 만듭니다. 이때 `file` 옵션으로 같은 폴더에 있는 lets get it.png 파일(확장자까지 적어야 합니다)을 불러옵니다. 그 다음 레이블 객체인 `img_label`을 생성하며 `image` 옵션으로 레이블에 띄울 이미지 객체를 설정합니다. `pack()` 메서드를 호출해 이미지 레이블을 창에 표시합니다.

기존보다 커진 화면에 추가한 이미지가 잘 나옵니다. `PhotoImage`는 GIF, PNG, PGM, PPM 형식만 사용할 수 있습니다. 이미지 파일을 읽을 수 없다는 오류가 발생하면 파일의 확장자를 확인해 보세요!

이처럼 `Label`로 객체를 만들 때 `text`를 설정하면 문자를, `image`를 설정하면 이미지를 나타낼 수 있습니다.

`NOTE` 알 수 없는 오류 발생 시

이미지 레이블을 만들 때 다음처럼 `TclError`가 발생하진 않았나요? 심지어 코드를 올바르게 작성했는데도요.

> `실행결과`
> ```
> ----> 6 img_label = Label(root, image=letsgetit)
>
> TclError: image "pyimage4" doesn't exist
> ```

이전 코드를 실행했을 때 오류가 발생하면 올바른 코드로 수정하더라도 해당 코드에서 오류가 발생합니다. 재실행해도 `pyimage` 뒤의 숫자만 증가할 뿐이죠. 해결책은 간단합니다. 에디터 프로그램의 커널을 재실행하면 됩니다. 주피터 노트북에서는 상단 [RUN] 버튼 옆의 [restart] 버튼을 클릭하세요.

창에 띄운 이미지가 조금 커 보이니 이미지 크기를 줄여 봅시다. 이미지 크기는 `PhotoImage` 클래스의 `subsample()` 메서드로 조절합니다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('800x400')

letsgetit = PhotoImage(file='lets get it.png')
letsgetit = letsgetit.subsample(2, 2)
img_label = Label(root, image=letsgetit)
img_label.pack()

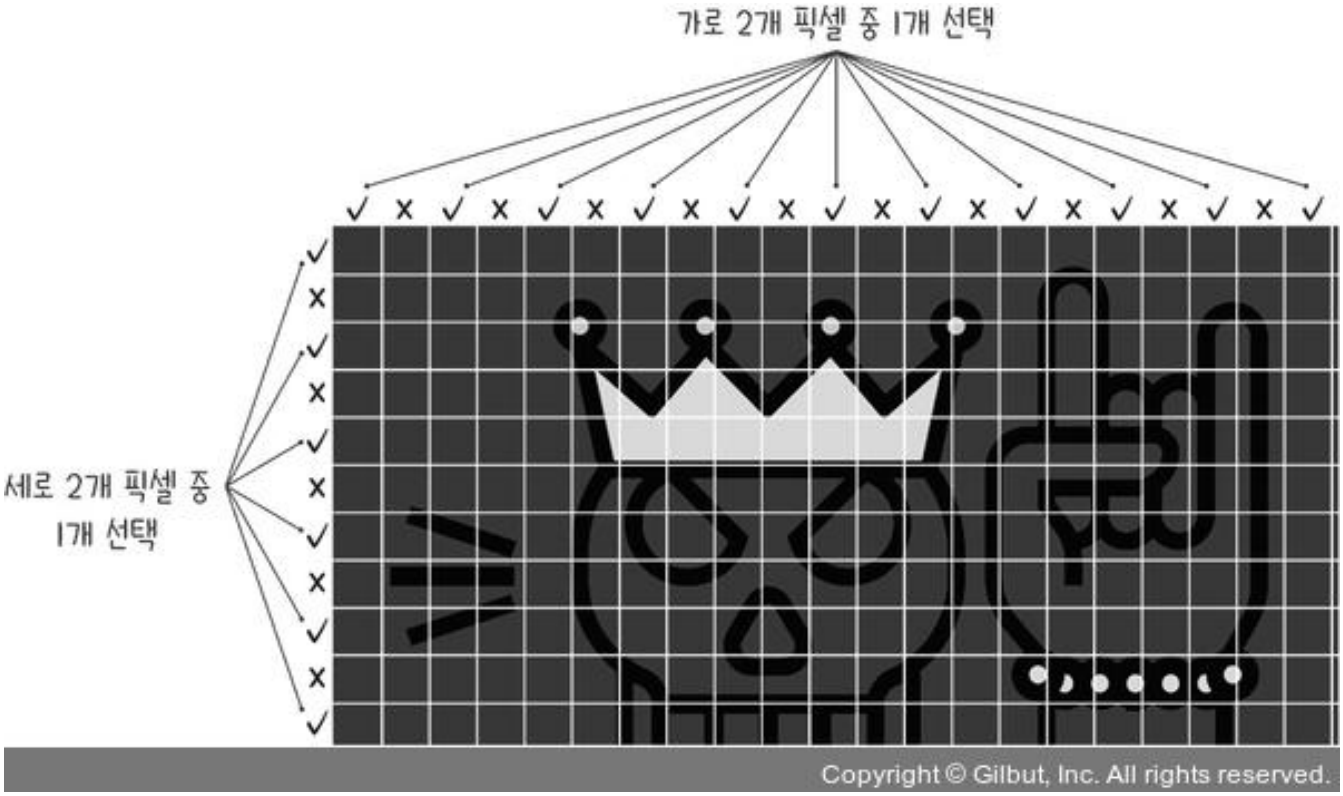
root.mainloop()
```

실 행 결 과



이미지 크기가 줄어듭니다. `letsgetit` 객체의 크기를 `letsgetit.subsample(2, 2)`로 줄이고 이를 다시 `letsgetit`에 저장했습니다.

`subsample()`은 그림의 n개 픽셀 중 1개 픽셀을 선택해 그림 크기를 줄일 때 사용합니다. 이미지는 보통 픽셀로 이루어져 있는데 크기를 줄이려면 픽셀의 크기를 줄이는 것이 아니라 픽셀 개수를 줄여야 합니다(픽셀 크기는 고정입니다). 그런데 전체 그림의 형태가 훼손되도록 그림 한 쪽의 픽셀만 선택하거나 제거하면 안 되겠죠. 그래서 균일하게 n번째 픽셀만 제거하는 겁니다. `letsgetit.subsample(2, 2)`는 가로, 세로 모두 픽셀을 2개마다 하나씩 제거하고 하나만 선택한다는 의미입니다. 그림으로 표현하면 다음과 같습니다. 이렇게 되면 가로와 세로 길이가 각각 1/2로 줄어듭니다.



▲ 그림 15-3 픽셀 줄이기

가령 `subsample(5, 3)`은 가로 픽셀은 다섯 번째마다, 세로 픽셀은 세 번째마다 선택하므로 가로는 1/5, 세로는 1/3로 줄어듭니다.

15.2.2 위젯 배치하기

가장 기본 위젯인 레이블을 배웠습니다. 보통 위젯은 단독으로 있는 경우가 많지 않고 대부분 여러 위젯이 모여서 기능합니다. 그래서 다른 레이블을 배우기 전에 여러 위젯을 배치하는 방법을 먼저 배워 보겠습니다.

앞의 코드에서 봤듯이 위젯을 창에 표시하려면 생성한 위젯마다 위젯을 배치하는 명령어(`pack()`)를 설정해야 합니다. 그렇지 않으면 위젯이 화면에서 표시되지 않습니다. `tkinter`에서 위젯을 배치할 때는 절대위치를 활용하는 방법과 상대위치를 활용하는 방법이 있습니다.

절대위치로 배치하기

절대위치는 다음과 같이 위젯이 표시될 위치의 좌표를 숫자로 정확하게 제시하는 방법입니다.

형식

```
place(x=x좌표, y=y좌표)
```

값을 넣어 실행해 봅시다.

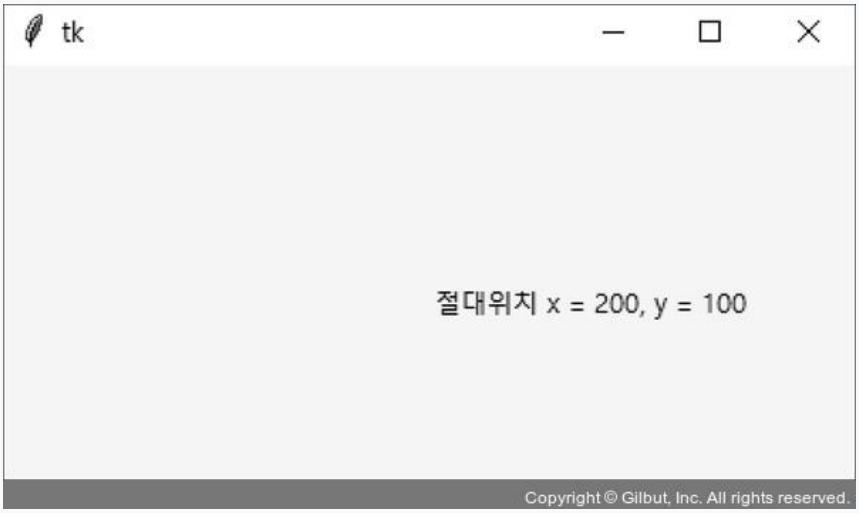
```
from tkinter import *

root = Tk()
root.geometry('400x200')

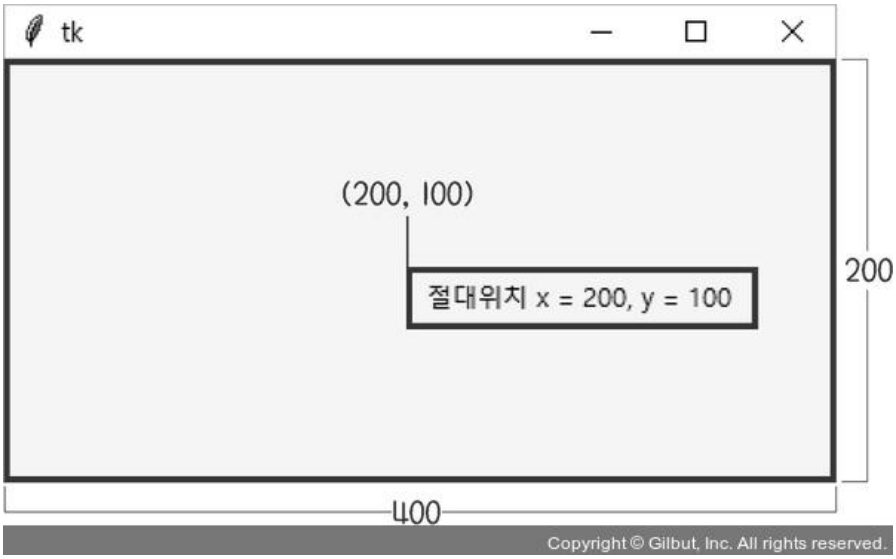
label1 = Label(root, text='절대위치 x = 200, y = 100')
label1.place(x=200, y=100)

root.mainloop()
```

실행결과



레이블 객체 `label1`을 생성하면서 위치를 (200, 100)으로 설정했습니다. 화면 크기가 너비 400픽셀, 높이 200픽셀이므로 텍스트 레이블의 왼쪽 상단 모서리 부분이 `place()`로 설정한 위치에 해당함을 알 수 있습니다.



▲ 그림 15-4 위젯의 절대위치 기준점

상대위치로 배치하기: pack()

절대위치는 위치를 세밀하게 정할 수 있다는 장점이 있으나 위젯 위치를 일일이 지정해야 하므로 위젯이 많은 경우에는 코드가 복잡합니다. 그래서 상대위치를 사용합니다.

상대위치는 위치 좌표 대신에 위젯 간 상대적 위치로 위젯을 배치하는 방법입니다. 예를 들어, '위젯 A는 위젯 B보다 위에 있어야 하고, 위젯 C는 위젯 B의 오른쪽에 있어야 한다'는 기준으로 위치를 설정하는 것이죠. `tkinter`에서 상대위치를 설정하는 방법은 앞에서 배운 `pack()`과 뒤에 나올 `grid()`가 있습니다. `pack()`부터 살펴봅시다.

`pack()`은 별다른 위치를 설정하지 않으면 창 중앙에 위에서 아래의 순서로 위젯을 배치합니다. 위젯을 구분할 수 있게 `groove` 옵션으로 테두리 모양을 설정한 레이블 4개를 만든 후 `pack()`으로 배치해 봅시다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200')

label1 = Label(root, text='안녕하세요!', relief='groove')
label1.pack()
label2 = Label(root, text='Hello!', relief='groove')
label2.pack()
label3 = Label(root, text='니하오!', relief='groove')
label3.pack()
label4 = Label(root, text='봉쥬르!', relief='groove')
label4.pack()

root.mainloop()
```

실행결과



4개의 레이블을 만들고, 각 레이블의 위치를 각각 `label1.pack()`, `label2.pack()`, `label3.pack()`, `label4.pack()`으로 정합니다. 위젯을 만들고 배치하는 명령어가 없으면 창에 표시되지 않으므로 모든 위젯에서 `pack()`을 실행해야 합니다. 위치를 설정하지 않아 창 중앙에 순서대로 배치됐습니다.

이번에는 `pack()`에 위치를 설정하는 옵션을 추가해 보겠습니다. 위치를 설정하는 옵션은 `side`이며, `left`, `right`, `top`, `bottom`을 옵션 값으로 넣을 수 있습니다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200')

label1 = Label(root, text='안녕하세요!', relief='groove')
label1.pack(side='left')
label2 = Label(root, text='Hello!', relief='groove')
label2.pack(side='right')
label3 = Label(root, text='니하오!', relief='groove')
label3.pack(side='top')
label4 = Label(root, text='봉쥬르!', relief='groove')
label4.pack(side='bottom')

root.mainloop()
```

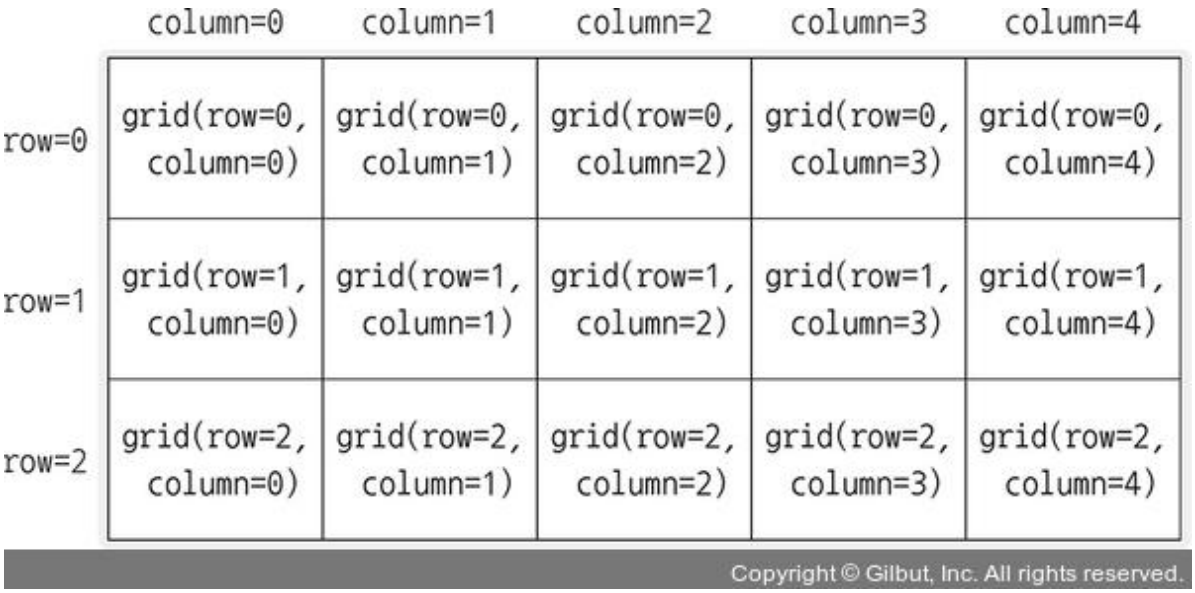
실행결과



레이블이 사방으로 배치됩니다. 이처럼 `pack()`은 창 내에서 상대위치를 활용해 위젯을 정렬합니다.

상대위치로 배치하기: grid()

위젯을 상대위치로 배치하는 다른 방법으로 `grid()`가 있습니다. `grid()`는 이름 그대로 격자를 만들어 격자 한 칸에 위젯을 배치하는 방법입니다. `grid()`는 배치할 위치를 `row`와 `column` 옵션으로 지정해야 합니다. `row`는 행, `column`은 열을 의미하죠.



▲ 그림 15-5 grid()에 옵션주기

그림처럼 창을 지정한 행과 열 수로 나눈 후에 위젯을 배치하려는 위치의 행과 열을 `row`와 `column` 옵션에 숫자로 지정합니다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200')

label1 = Label(root, text='안녕하세요!', relief='groove')
label1.grid(row=0, column=0)
label2 = Label(root, text='Hello!', relief='groove')
label2.grid(row=0, column=1)
label3 = Label(root, text='니하오!', relief='groove')
label3.grid(row=1, column=0)
label4 = Label(root, text='봉쥬르!', relief='groove')
label4.grid(row=1, column=1)

root.mainloop()
```

실행결과


my first tkinter
안녕하세요!
Hello!
니하오!
봉쥬르!
Copyright © Gilbut, Inc. All rights reserved.

지정한 행과 열에 각 위젯이 나타납니다. 창에 표시된 위젯의 행과 열은 다음 그림과 같습니다.



▲ 그림 15-6 위젯의 상대위치 배치

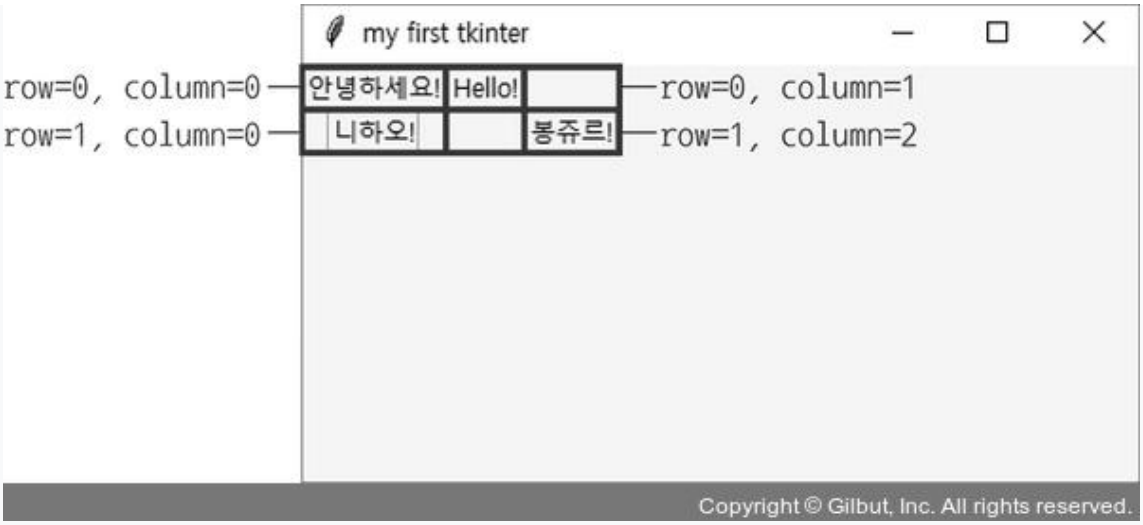
`grid()`는 특히 위젯이 여러 개일 때 사용하기 편합니다. 격자에서 칸에 맞는 숫자만 적어 주면 되기 때문이죠. 그런데 레이블이 너무 붙어 있어서 중간에 공간을 넣는다고 해 봅시다. 두 번째 행의 위치를 `(row=9, column=0)`, `(row=9, column=9)`로 설정하면 될까요?

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200')

label1 = Label(root, text='안녕하세요!', relief='groove')
label1.grid(row=0, column=0)
label2 = Label(root, text='Hello!', relief='groove')
label2.grid(row=0, column=1)
label3 = Label(root, text='니하오!', relief='groove')
label3.grid(row=9, column=0)
label4 = Label(root, text='봉쥬르!', relief='groove')
label4.grid(row=9, column=9)

root.mainloop()
```

실행결과


my first tkinter
row=0, column=0
row=1, column=0
안녕하세요!
Hello!
니하오!
봉쥬르!
row=0, column=1
row=1, column=2
Copyright © Gilbut, Inc. All rights reserved.

공간이 의도한 대로 벌어지지 않습니다. `label3`은 그대로 `(row=1, column=0)` 위치에, `label4`는 `(row=1, column=2)`에 배치됐습니다.

이런 결과가 나오는 이유는 `grid()`의 특성 때문입니다. `grid()`는 `row`와 `column`에 숫자가 차례대로 나오지 않으면 직전에 나온 숫자의 바로 다음 숫자를 자동으로 설정합니다. 코드에서 `row`는 0까지 나왔기 때문에 `(row=9, column=0)`으로 설정해도 자동으로 `row`의 값을 9에서 1로 바꿔 위젯을 배치합니다. `column`은 그 전에 0이 등장한 적이 있으므로(`label1`) 지정한 위치 0에 배치하고요. `label4.grid(row=9, column=9)`도 마찬가지입니다. 이전에 `row`는 0까지 나타났으므로 1로, `column`은 1까지 나타났으므로 2로 변경해 화면에 배치합니다.

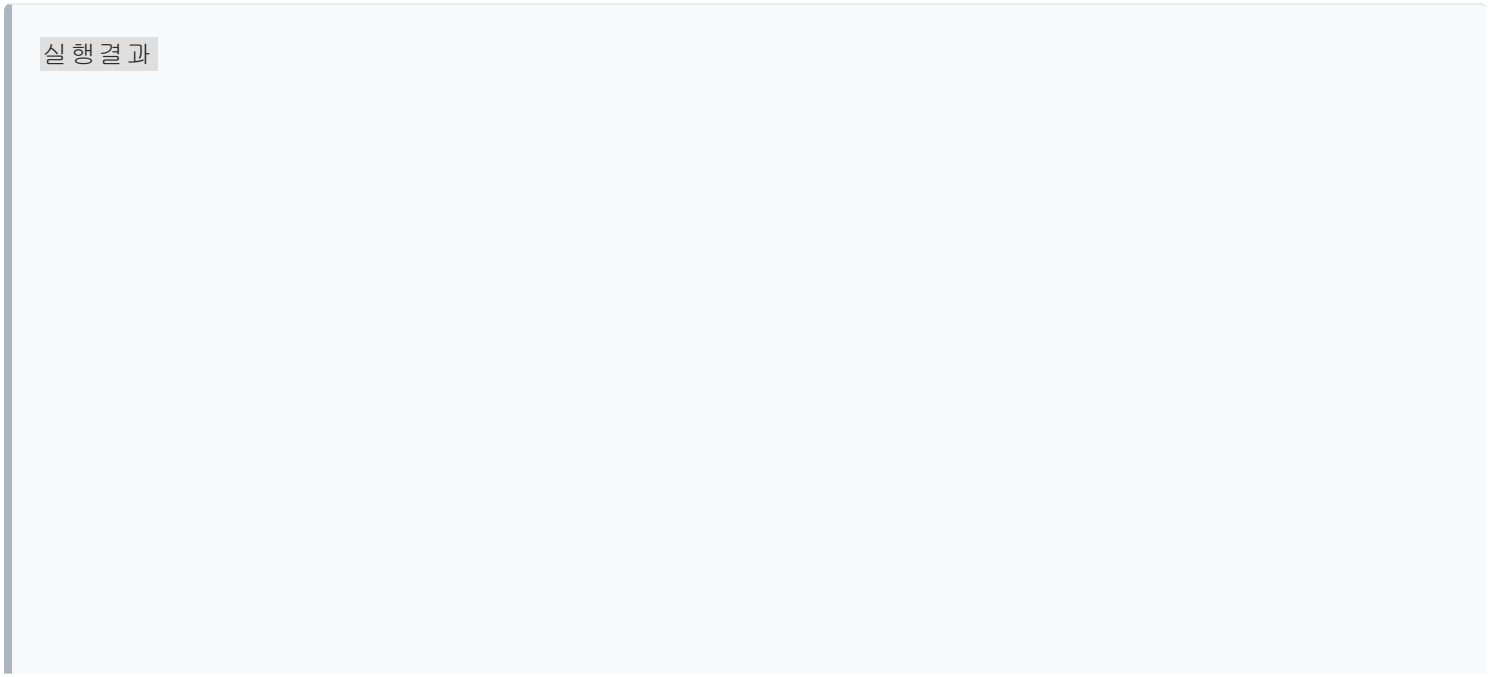
이번에는 위젯의 크기를 설정하고 `grid()`로 어떻게 표현되는지 살펴봅시다.

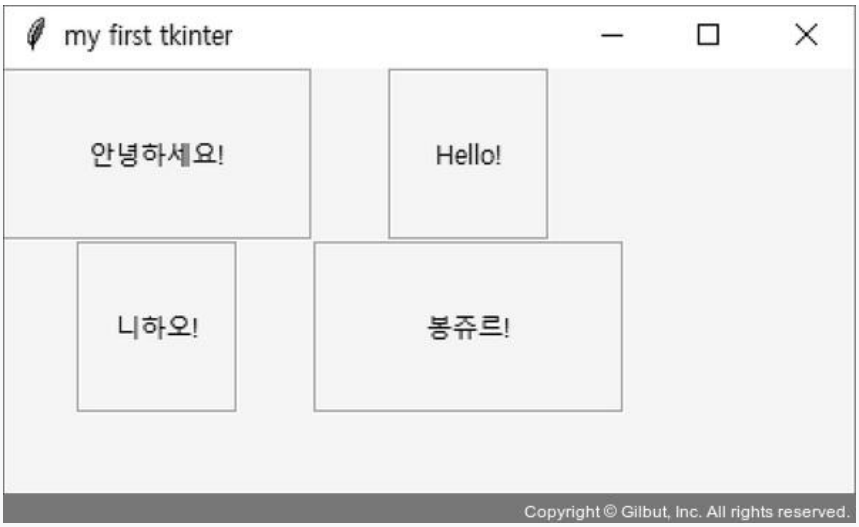
```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200')

label1 = Label(root, text='안녕하세요!', width=20, height=5, relief='groove')
label1.grid(row=0, column=0)
label2 = Label(root, text='Hello!', width=10, height=5, relief='groove')
label2.grid(row=0, column=1)
label3 = Label(root, text='니하오!', width=10, height=5, relief='groove')
label3.grid(row=1, column=0)
label4 = Label(root, text='봉쥬르!', width=20, height=5, relief='groove')
label4.grid(row=1, column=1)

root.mainloop()
```

실행결과




my first tkinter
안녕하세요!
Hello!
니하오!
봉쥬르!
Copyright © Gilbut, Inc. All rights reserved.

설정한 위젯 크기에 맞게 격자 크기도 커졌습니다. 결과를 보면 각 줄의 가장 긴 길이에 맞춰진 것이 보입니다. `label1`의 너비가 20이므로 같은 열에 있는 `label3`도 20만큼의 공간을 확보합니다. 단 `label3`은 너비가 10이므로 해당 칸을 꽉 채우지 못하고 중간에 위치합니다. `label2`와 `label4`도 마찬가지로 더 긴 칸의 너비만큼 공간을 배치했습니다.

이처럼 위젯을 생성할 때 너비와 높이를 설정해서 `grid()`를 사용하면 원하는 모양에 가깝게 표현할 수 있습니다.

NOTE 크기 단위

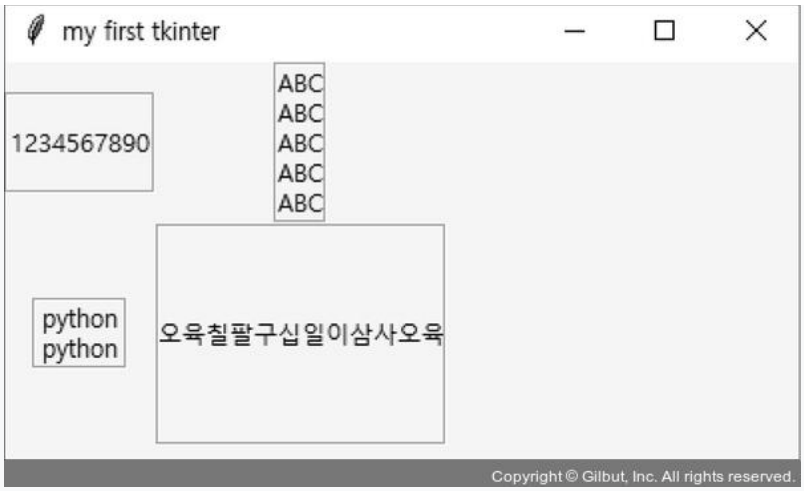
창 크기를 설정할 때는 픽셀을 단위로 사용했지만, 위젯의 `width`와 `height`는 한 글자의 너비와 높이를 각각의 단위로 사용합니다. 예를 들어 `weight=10`은 문자(숫자 혹은 알파벳)가 10개 들어갈 너비를 의미하고, `height=3`은 문자가 3개 들어갈 수 있는 높이를 의미합니다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200')

label1 = Label(root, text='1234567890', width=10, height=3, relief='groove')
label1.grid(row=0, column=0)
label2 = Label(root, text='ABC\nABC\nABC\nABC\nABC', width=3, height=5,
relief='groove')
label2.grid(row=0, column=1)
label3 = Label(root, text='python\npython', width=6, height=2, relief='groove')
label3.grid(row=1, column=0)
label4 = Label(root, text='일이삼사오육칠팔구십일이삼사오육칠팔구십', width=20,
height=7, relief='groove')
label4.grid(row=1, column=1)

root.mainloop()
```

실행결과



`ABC\nABC\nABC\nABC\nABC`에 나오는 `\n`은 줄바꿈을 의미합니다. ABC 출력 후 4번 줄바꿈하고 마지막 줄에 ABC를 출력하므로 ABC가 총 5줄 출력됩니다. 화면 결과를 보면 `width`와 `height`가 숫자 또는 알파벳 한 자를 기준으로 길이를 설정하는 것을 확인할 수 있습니다. `width=10`, `height=3`인 `label1`은 10개 숫자가 `label1`의 가로 공간을 꽉 채웁니다. `label2`는 `width=3`, `height=5`인데, 다섯 줄의 `ABC`가 세로 공간을 꽉 채웠습니다. `label3`도 마찬가지입니다.

한글을 넣은 `label4`를 봅시다. 한글 20자를 넣었는데 앞뒤로 글자가 잘려서 표시됩니다. 이는 숫자나 알파벳 한 자보다 한글 한 자의 너비가 넓어서 그렇습니다. `width`, `height`를 사용할 때는 이 부분을 유의하세요.

이번에는 격자의 여러 칸을 합칠 때 사용하는 `rowspan`과 `columnspan`을 살펴보겠습니다. `rowspan`, `columnspan`은 엑셀에서 사용하는 셀 병합 기능과 비슷합니다. `rowspan=n`은 n개 행을 병합하고, `columnspan=n`은 n개 열을 병합합니다.

형식

```
rowspan=n      # n은 숫자
columnspan=n   # n은 숫자
```

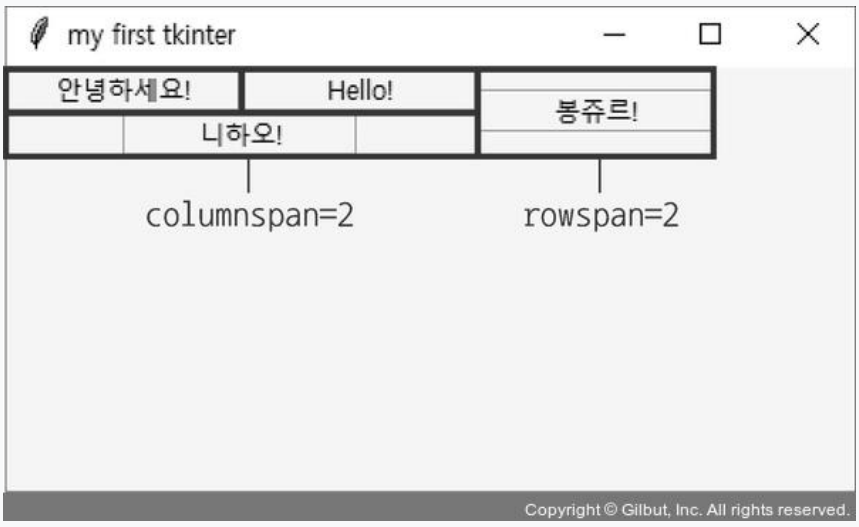
코드에 적용해 봅시다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('400x200')

label1 = Label(root, text='안녕하세요!', width=15, relief='groove')
label1.grid(row=0, column=0)
label2 = Label(root, text='Hello!', width=15, relief='groove')
label2.grid(row=0, column=1)
label3 = Label(root, text='니하오!', width=15, relief='groove')
label3.grid(row=1, column=0, columnspan=2)
label4 = Label(root, text='봉쥬르!', width=15, relief='groove')
label4.grid(row=0, column=2, rowspan=2)

root.mainloop()
```

실행결과



`label3`에 `columnspan=2`를 설정하니 가로 두 칸이 병합된 위치에 레이블이 배치됩니다. `label3`의 `width`는 15이기 때문에 병합된 칸을 모두 채우지 않고 중앙에 있습니다. `label4`도 `rowspan=2`를 설정하니 세로 두 칸을 병합하고 중앙에 레이블을 배치합니다. 각 레이블의 너비와 높이를 조절하면 병합한 칸의 크기에 딱 맞게 위젯을 만들 수 있습니다.

`grid()`를 활용해 텍스트와 이미지를 함께 창에 띄울 수도 있습니다.

```
root = Tk()
root.title('my first tkinter')
root.geometry('700x450')

label_txt = Label(root, text='파이썬 프로그래밍', font=('Malgun Gothic', 30))
label_txt.grid(row=0, column=0)
letsgetit = PhotoImage(file='lets get it.png')
label_img = Label(root, image=letsgetit)
label_img.grid(row=1, column=0)

root.mainloop()
```

실행결과



앞에서 배운 내용을 조합해 코드를 작성했으니 자세한 설명은 생략하겠습니다. 본인이 원하는 이미지와 텍스트를 조합해 다양한 화면을 만들어 보세요!

15.2.3 버튼

이번에는 `tkinter`에서 버튼(Button) 만드는 방법을 알아보겠습니다. 버튼 역시 `tkinter`에 미리 정의되어 있는 클래스입니다. 클래스의 생성자 옵션은 `help()`를 실행해 직접 확인해 보세요.

버튼 만들기

레이블을 만들 때처럼 버튼이 위치할 창과 `Button` 클래스의 객체를 만듭니다. 버튼에 표시할 글자는 `text` 옵션으로 넣고, 버튼의 크기 역시 레이블처럼 `width`와 `height`로 설정합니다.

```
from tkinter import *

root = Tk()
root.title('Button')
root.geometry('400x200')

btn1 = Button(root, text='버튼', width=10, height=3)
btn1.pack()

root.mainloop()
```

실행결과



`Button`의 `btn1` 객체를 만듭니다. 창에 `btn1` 객체가 나타나고, `text` 옵션으로 설정한 텍스트(버튼)가 표시됩니다. 이때 버튼을 클릭하면 아무 일도 일어나지 않습니다. 버튼을 클릭했을 때 실행할 내용을 작성하지 않았기 때문이죠.

이 부분은 뒤에서 배우고 먼저 다른 위젯과 버튼을 `grid()`로 창에 배치해 보겠습니다.

```
root = Tk()
root.title('Button')
root.geometry('400x200')

lbl1 = Label(root, text='안녕하세요!', width=40)
lbl1.grid(row=0, column=0)
btn1 = Button(root, text='확인', width=10)
btn1.grid(row=0, column=1)

root.mainloop()
```

실행결과



텍스트 레이블 옆에 버튼이 나타납니다. 투박하긴 하지만, 이런 식으로 여러 위젯을 함께 배치할 수 있습니다.

버튼 클릭 시 실행될 이벤트 추가하기

그럼 버튼을 클릭했을 때 어떤 반응이 나타나게 만들어 봅시다. 버튼을 클릭했을 때 일어나는 일을 이벤트라고 합니다. 이벤트는 13장에서 배운 사용자 정의 함수를 활용해 만듭니다. 그리고 버튼 객체를 만들 때 `command` 옵션을 사용해 함수명을 지정하면 해당 버튼을 클릭할 때마다 `command`에 지정한 함수가 실행됩니다.

```
def sayhello():
    print('Hello!! ' * 3)

root = Tk()
root.title('Button')
root.geometry('400x200')

lbl1 = Label(root, text='안녕하세요! ', width=40)
lbl1.grid(row=0, column=0)
btn1 = Button(root, text='확인', width=10, command=sayhello)
btn1.grid(row=0, column=1)

root.mainloop()
```

실행결과




```
Hello!! Hello!! Hello!!
```

`sayhello()`라는 함수를 만들고, 버튼 객체를 생성할 때 `command` 옵션에 `sayhello` 함수를 넣었습니다. command는 '명령하다'라는 뜻이죠. 단어 뜻 그대로 버튼을 클릭할 때 실행될 명령어가 바로 함수 `sayhello()`라는 의미입니다. `sayhello()`는 `'Hello!! '`를 3번 출력하는 함수입니다.

확인 버튼을 클릭해도 `tkinter` 화면에는 아무 일도 벌어지지 않습니다. 그런데 결과창에 'Hello!! '가 3번 출력됩니다. 왜 `tkinter` 화면이 아닌 결과창에 출력될까요?

`print()`는 화면이 아닌 결과창에 출력하는 명령어입니다. 지금까지 `tkinter` 화면에 텍스트를 나타내려면 레이블을 만들고 `text` 옵션으로 레이블에 표시할 텍스트를 설정했습니다. 배운 대로라면 다음처럼 만듭니다.

```
lbl2 = Label(root, text='Hello!! Hello!! Hello!! ')
lbl2.grid(row=1, column=0)
```

실 행 결 과



기존 코드에 앞의 코드를 더해 실행해 보면 오류는 발생하지 않지만, 의도치 않은 문제가 생깁니다. 버튼을 클릭하지 않아도 텍스트가 보이죠. 버튼을 클릭할 때 텍스트가 보이게 하려면 화면을 처음 생성할 때 두 번째 레이블(`lbl2`)에는 어떤 글자도 보이지 않아야 합니다. 그리고 버튼을 클릭했을 때만 텍스트가 보여야 하죠. 따라서 `lbl2` 객체에 들어가는 값이 화면을 생성할 때와 버튼을 클릭할 때에 달라야 합니다.

이처럼 화면이 생성된 후에 변하는 값을 저장하는 변수를 동적변수라고 합니다. 동적변수는 화면이 만들어진 이후 다양한 이벤트에 의해 값이 변합니다. 동적변수도 변수이므로 사용하려면 먼저 만들어야 합니다. 동적변수의 값이 문자열일 때는 `StringVar()`, 정수형일 때는 `IntVar()`로 만듭니다. 그리고 실수형은 `float()`가 아닌 `DoubleVar()`로 만듭니다.

형식

```
변수명 = StringVar() # 또는 IntVar(), DoubleVar()
```

그리고 객체에 `textvariable` 옵션을 줘서 사용할 동적변수를 지정합니다.

형식

```
레이블명 = Label(root, textvariable=동적변수)
```

동적변수를 만들고 실행해 보면 동적변수가 들어 있는 공간은 아무것도 없는 빈 공간으로 보입니다. 이런 동적변수의 값을 변경하고 싶을 때 `set()`을 사용합니다. `set()`에 변경하려는 값을 넣고 실행하면 동적변수 위치에 변경하려는 값이 나타납니다.

형식

```
동적변수.set(변경값)
```

앞의 코드를 동적변수를 사용하는 방식으로 수정해 봅시다.

```
# ----- 프로그램 기능 부분 -----
def sayhello():
    txt.set('Hello!! ' * 3) # sayhello() 함수 실행 시 동적변수 txt에 값 넣기

# ----- 프로그램 화면 부분 -----
root = Tk()
root.title('Button')
root.geometry('400x200')

lbl1 = Label(root, text='안녕하세요! ', width=40)
lbl1.grid(row=0, column=0)
btn1 = Button(root, text='확인', width=10, command=sayhello)
btn1.grid(row=0, column=1)
txt = StringVar() # 동적변수 txt 정의
lbl2 = Label(root, textvariable=txt, width=40, height=3) # 레이블에 동적변수 txt 설정
lbl2.grid(row=1, column=0, columnspan=2)

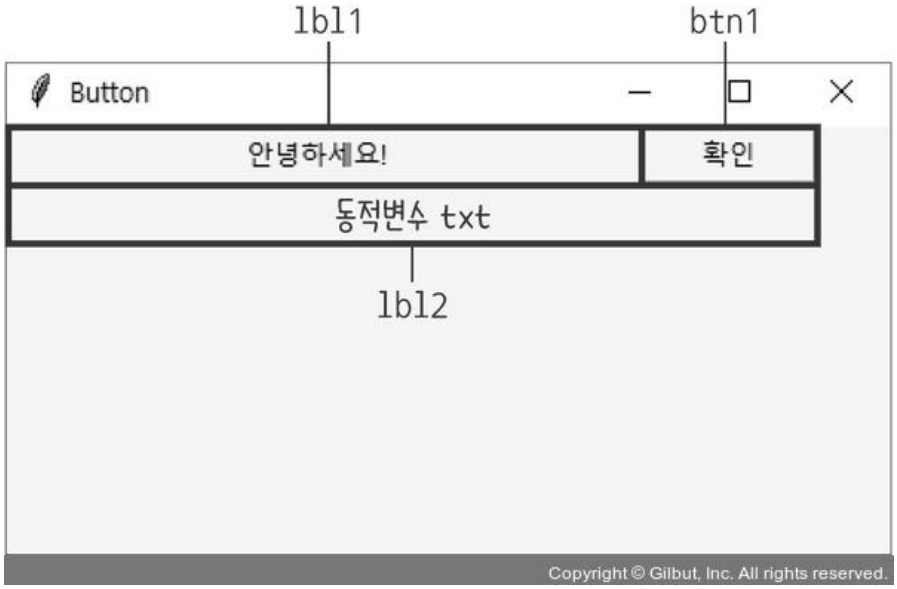
root.mainloop()
```

실행결과



Copyright © Gilbut, Inc. All rights reserved.

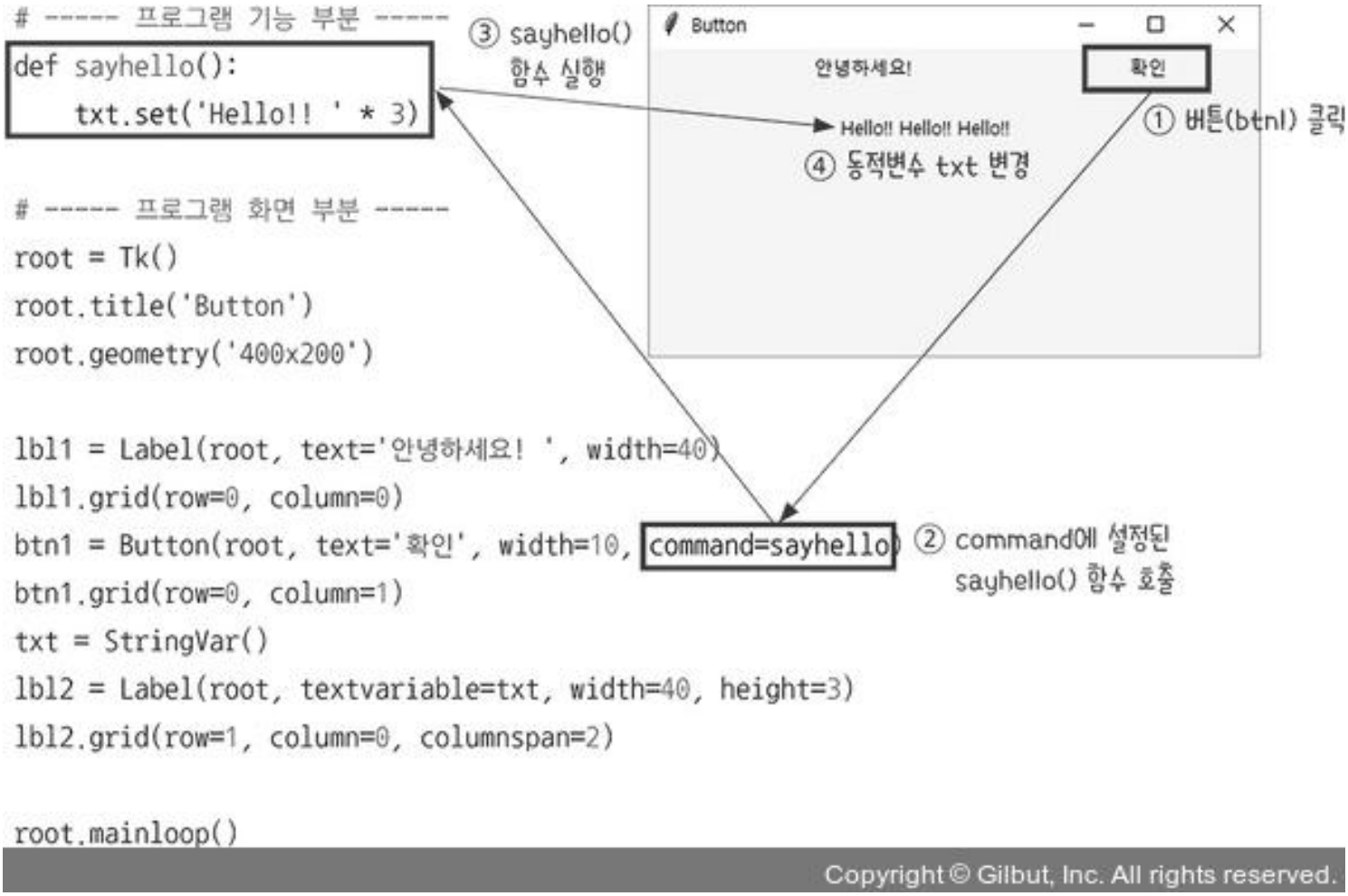
프로그램 화면 부분에 추가된 코드부터 봅시다. `StringVar()`로 문자를 받는 동적변수 `txt`를 만들었습니다. 레이블 객체를 생성할 때 `textvariable` 옵션으로 동적변수 `txt`를 설정했습니다. 레이블의 크기와 위치도 적절하게 추가했고요. 동적변수는 레이블을 생성하기 전에 만들어야 레이블의 `textvariable` 옵션에 설정할 수 있습니다. 이를 그림으로 표현하면 다음과 같습니다. 레이블 `lbl2` 안에 동적변수 `txt`가 들어 있는 형태입니다.



▲ 그림 15-7 레이블에 설정된 동적변수

`sayhello()` 함수도 수정했습니다. 두 번째 레이블인 `lbl2` 객체는 처음에 아무것도 없다가 버튼을 클릭하고 나면 'Hello!! Hello!! Hello!! '로 변경돼야 합니다. `sayhello()` 함수는 버튼을 클릭했을 때 실행될 이벤트이므로 `set()`에 변경될 `txt`의 값 `'Hello!! ' * 3`을 넣습니다.

코드를 실행하면 초기 화면은 이전과 동일합니다. 확인 버튼을 누르면 화면에 'Hello!! Hello!! Hello!! '가 나타납니다. 이때 코드가 실행되는 순서는 다음과 같습니다.



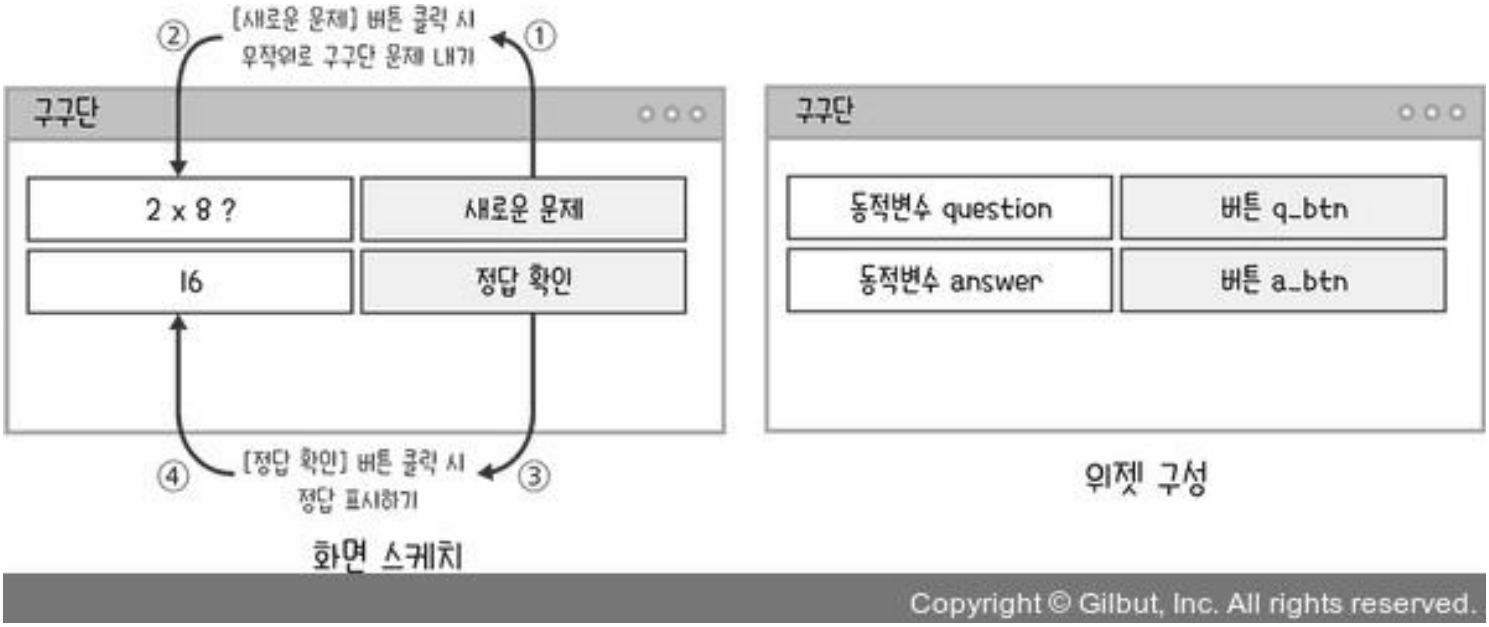
Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 15-8 버튼 클릭 시 코드 실행 순서

버튼을 클릭하면 `command`로 설정된 `sayhello()` 함수가 호출됩니다. 호출된 `sayhello()` 내부에서 `txt.set('Hello!! ' * 3)`이 실행되면 `lbl2` 객체에 설정된 `txt`의 값이 변경되어 레이블에 표시됩니다.

버튼으로 구구단 프로그램의 화면 만들기

배운 내용을 활용해 구구단 프로그램의 화면을 작성해 봅시다. 구구단 문제는 반복문을 배울 때부터 꾸준히 접했기 때문에 이제 익숙해졌을 겁니다. 화면은 다음처럼 구성합니다.



▲ 그림 15-9 구구단 프로그램 화면 구성

첫 번째 행의 오른쪽에 있는 [새로운 문제] 버튼을 클릭하면 첫 번째 행 왼쪽에 문제가 나타납니다. 그리고 두 번째 행의 오른쪽에 있는 [정답 확인] 버튼을 클릭하면 두 번째 행의 왼쪽에 정답이 나타납니다. [새로운 문제] 버튼을 클릭할 때마다 이전과는 다른 문제가 나와야 합니다. 따라서 새로운 문제가 나타나는 위치에는 화면이 생성된 후 값이 변하는 동적변수를 사용합니다. 답이 나타나는 부분도 동일합니다. 문제가 변하면 정답도 변해야 하므로 동적변수를 사용합니다.

정리한 내용을 코드로 작성해 봅시다. 프로그램 화면 부분부터 작성해 보겠습니다.

```
from tkinter import *

root = Tk()
root.title('구구단')

question = StringVar() # 무작위로 생성된 구구단 문제를 저장하는 동적변수
q_lbl = Label(root, textvariable=question, width=30) # 구구단 문제가 나타나는 레이블
q_lbl.grid(row=0, column=0)

q_btn = Button(root, text='새로운 문제', width=10)
q_btn.grid(row=0, column=1)

a_btn = Button(root, text='정답 확인', width=10)
a_btn.grid(row=1, column=1)

answer = StringVar() # 구구단 문제의 정답을 저장하는 동적변수
a_lbl = Label(root, textvariable=answer, width=30) # 구구단 정답이 나타나는 레이블
a_lbl.grid(row=1, column=0)

root.mainloop()
```

실행결과



프로그램 화면 구성은 어렵지 않습니다. 구구단 프로그램을 띄울 창을 만듭니다. 그리고 문제와 정답을 출력할 레이블, 새로운 문제를 출력할 버튼과 정답을 확인할 버튼 4가지 위젯을 만들면 됩니다.

먼저 2개의 레이블은 `q_lbl`과 `a_lbl`로 만듭니다. 이때 레이블에서 동적변수를 사용하므로 동적변수를 레이블보다 먼저 만들어야 합니다. `StringVar()`로 바뀌는 문제를 저장할 동적변수 `question`과 문제에 따라 바뀌는 정답을 저장할 `answer`를 만듭니다.

> TIP `answer`를 `IntVar()`로 정의하면 처음에 자동으로 0을 저장하게 되어 실행한 직후 정답 레이블에 0을 표시합니다. 그래서 동적변수를 `StringVar()`로 만듭니다.

첫 번째 레이블은 버튼을 클릭할 때마다 문제가 바뀌어야 하므로 `textvariable=question`으로 동적변수를 설정해 문제를 가져옵니다. 두 번째 레이블도 버튼을 클릭할 때마다 정답이 바뀌어야 하므로 `textvariable=answer`로 동적변수를 설정해 정답을 가져옵니다. 버튼 2개도 각각 `q_btn`, `a_btn`으로 만듭니다. 레이블 크기는 너비 30, 버튼 크기는 너비 10으로 통일했고, 높이는 둘 다 설정하지 않았습니다. `root` 창의 크기도 설정하지 않았습니다. 창의 크기를 설정하지 않으면 자동으로 위젯 크기에 맞춥니다.

이제 프로그램 기능 부분을 하나씩 구현해 봅시다. [새로운 문제] 버튼을 클릭하면 동적변수 `question`의 값이 바뀌고, `question`의 값을 표시하는 `q_lbl` 레이블에 새로운 문제가 나타나야죠. `q_btn`을 클릭하면 실행되는 사용자 정의 함수 `make_question()`을 다음과 같이 만듭니다.

```
import random

# 무작위로 구구단 문제를 출제하는 함수
def make_question():
    num1, num2 = random.randint(2, 9), random.randint(1, 9)
    question.set(str(num1) + 'x' + str(num2) + '? ')
```

`random` 라이브러리를 추가하고 `make_question()` 함수에 무작위로 구구단 문제를 내는 코드를 작성합니다. 구구단 문제는 앞에서 여러 번 다루었으므로 여기서는 설명을 생략하겠습니다.
5장을 참고해 코드를 작성하고 마지막에 출제한 문제를 동적변수 `question`에 `set()` 함수로 설정하는 부분만 추가하면 됩니다.

[정답 확인] 버튼을 누르면 실행되는 이벤트도 작성해 봅시다. `a_btn`을 누르면 문제에 해당하는 답이 `a_lbl` 레이블에 나타나야 합니다. 구구단 문제의 정답은 `num1 * num2`이죠. `a_btn` 버튼을 누르면 정답인 `num1 * num2`를 동적변수 `answer`에 설정하는 사용자 정의 함수 `check_answer()`를 만듭니다.

```
def check_answer():
    answer.set(num1 * num2)
```

이제 만들어진 코드를 합치면 됩니다. 합치기 전에 `make_question()`과 `check_answer()` 함수에 사용하는 변수 `num1`, `num2`를 봅시다. `num1`, `num2`는 두 함수에서 공통으로 사용합니다. 특정 함수에서만 사용하는 지역변수로 만들면 다른 함수에서는 두 변수를 읽거나 쓸 수 없겠죠? 따라서 `num1`, `num2`는 함수 외부에 정의해 전역변수로 만들고, `make_question()`에서 `global` 키워드를 사용해 값을 변경할 수 있게 합니다.

설명대로 프로그램 기능 부분을 모두 작성하고 버튼에 두 함수를 연결합니다. 최종 코드는 다음과 같습니다(기존과 동일한 코드는 생략 표시했습니다).

```
from tkinter import *
import random

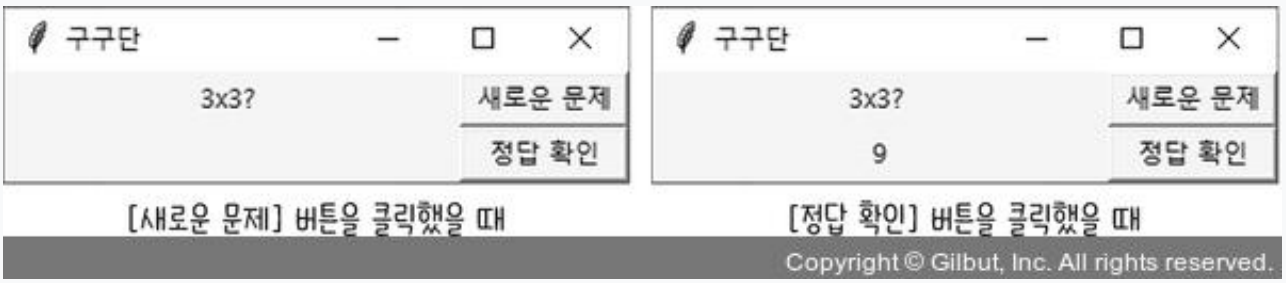
# ----- 프로그램 기능 부분 -----
num1, num2 = 0, 0 # 전역변수

def make_question():
    global num1, num2
    num1, num2 = random.randint(2, 9), random.randint(1, 9)
    question.set(str(num1) + 'x' + str(num2) + '? ')
def check_answer():
    answer.set(num1 * num2)

# ----- 프로그램 화면 부분 -----
…
q_btn = Button(root, text='새로운 문제', width=10, command=make_question)
q_btn.grid(row=0, column=1)

a_btn = Button(root, text='정답 확인', width=10, command=check_answer)
a_btn.grid(row=1, column=1)
…
```

실행결과



[새로운 문제] 버튼을 클릭하면 문제가 출제되고 [정답 확인] 버튼을 클릭하면 문제에 맞는 답이 나옵니다. 정상적으로 실행되지 않는다면 버튼에 `command`로 함수를 연결했는지 확인해 보세요. [새로운 문제] 버튼을 계속 클릭해서 이전과는 다른 문제가 출제되는지, [정답 확인] 버튼을 클릭하면 정답이 제대로 나오는지 확인해 보세요.

이처럼 버튼은 사용자 정의 함수와 동적변수를 적절히 활용해 다양한 기능을 구현할 수 있습니다.

15.2.4 엔트리

세 번째 위젯으로 화면에서 값을 입력받을 수 있는 엔트리(Entry)를 배워 보겠습니다. entry는 '들어옴', '입장' 등을 의미하죠. 단어의 의미처럼 엔트리를 사용하면 화면을 통해 들어오는 값을 프로그램에서 입력받을 수 있습니다.

엔트리 만들기

간단한 엔트리를 하나 만들어 봅시다.

```
from tkinter import *

root = Tk()
root.title('entry')
root.geometry('400x200')

ent = Entry(root, width=25)
ent.grid(row=0, column=0)

root.mainloop()
```

실 행 결 과



화면에 흰 칸이 하나 생겼습니다. 이렇게 생긴 입력칸이 바로 엔트리입니다. 엔트리 역시 `tkinter`에 `Entry` 클래스가 정의돼 있어서 클래스를 활용해 엔트리 객체를 만듭니다. 다른 위젯처럼 엔트리를 배치할 창(`root`)을 가장 먼저 만듭니다. 엔트리도 `width`, `height`로 너비와 높이를 지정합니다.

텍스트 레이블도 추가해서 다음과 같이 나타내 봅시다.

```
root = Tk()
root.title('entry')
root.geometry('400x200')

lbl = Label(root, text='이름 : ' , width=10)
lbl.grid(row=0, column=0)
ent = Entry(root, width=25)
ent.grid(row=0, column=1)

root.mainloop()
```

실행결과



엔트리 앞에 레이블을 추가해서 입력칸에 이름을 적도록 안내합니다. 입력칸에 텍스트를 입력해 보면 잘 입력됩니다.

입력된 텍스트를 프로그램으로 읽어 들여 봅시다. 엔트리에 입력된 값을 가져올 때는 `get()` 함수를 사용합니다. 버튼 하나를 추가하고, 버튼이 클릭될 때 엔트리에 입력된 값을 가져와 보겠습니다.

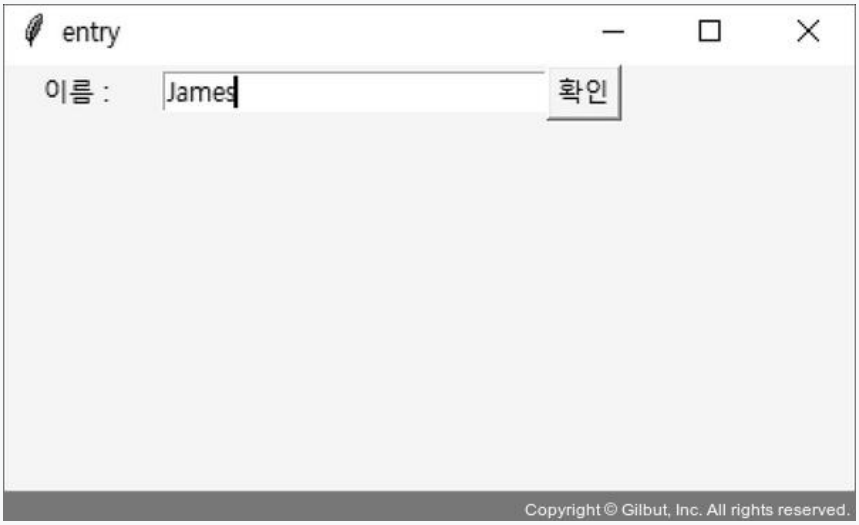
```
def entry_value():
    print(ent.get()) # 함수 실행 시 get()으로 ent의 값을 읽어 들여 출력함

root = Tk()
root.title('entry')
root.geometry('400x200')

lbl = Label(root, text='이름 : ' , width=10)
lbl.grid(row=0, column=0)
ent = Entry(root, width=25)
ent.grid(row=0, column=1)
btn = Button(root, text='확인', width=0, command=entry_value)
btn.grid(row=0, column=2)

root.mainloop()
```

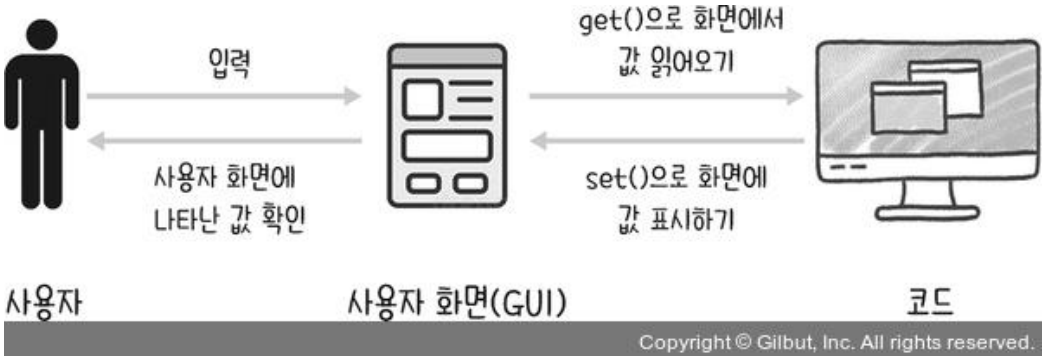
실행결과




```
James
```

버튼 객체 `btn`을 만들고 엔트리의 오른쪽에 배치했습니다(`(row=0, column=2)`). 버튼을 클릭하면 `command`로 연결된 `entry_value()`가 실행됩니다. 사용자 정의 함수 `entry_value()`는 `ent`에 입력된 값을 읽어와 결과창에 출력합니다. `ent`에 입력된 값은 `ent.get()`으로 읽어옵니다.

앞서 동적변수의 값을 창에 표시할 때는 `set()`을 사용했죠. `get()`은 `set()`과 반대로 창에 입력된 값을 읽어옵니다.

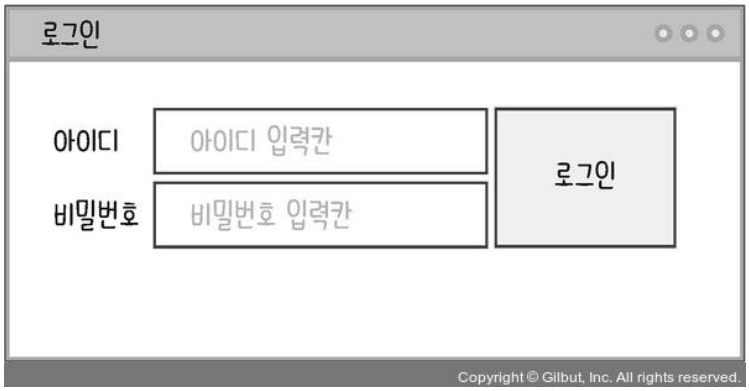


▲ 그림 15-10 get()과 set() 함수의 동작

[확인] 버튼을 누르면 창에서는 아무 일도 일어나지 않지만, 코드의 결과창에는 창에 입력한 이름이 출력됩니다. James를 입력하니 결과창에 James가 출력됩니다. 창에서 값을 제대로 읽어왔다는 것이죠.

로그인 화면 만들기

지금까지 배운 레이블, 버튼, 엔트리는 사용자 화면을 구성하는 가장 기본적인 요소입니다. 기본 요소들로 로그인 화면을 만들어 봅시다. 가장 흔하게 접하는 로그인 화면은 대부분 다음과 같은 형태입니다. 로그인 화면에서 아이디와 비밀번호를 입력하고 로그인 버튼을 누르면 회원으로 입장할 수 있습니다.



▲ 그림 15-11 로그인 화면

먼저 화면을 구성해 봅시다. 첫 번째 행에는 아이디와 관련한 텍스트 레이블과 엔트리, 두 번째 행에는 비밀번호와 관련한 텍스트 레이블과 엔트리가 배치되면 됩니다. 그리고 가장 오른쪽에는 두 행을 병합해 [로그인] 버튼을 만듭니다. 먼저 코드를 직접 작성해 본 후 책을 참고하세요.

```
# ----- 프로그램 화면 부분 -----
root = Tk()
root.title('Login')
root.geometry('400x200')

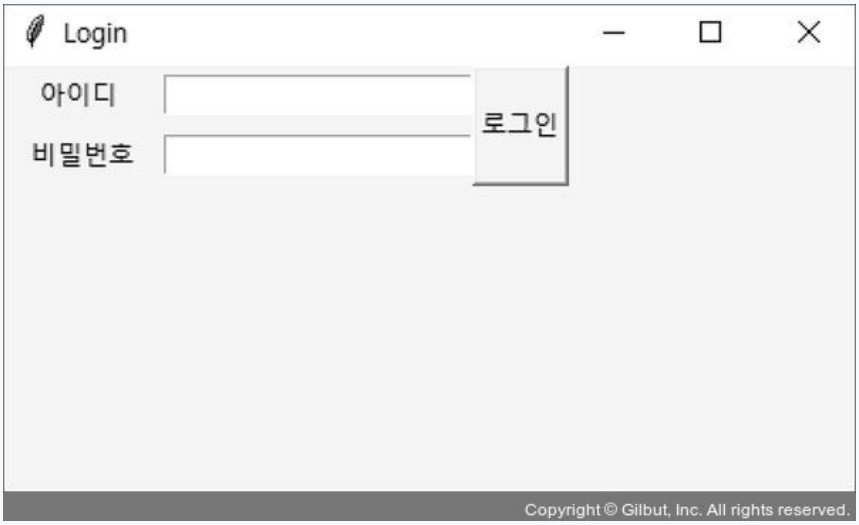
id_lbl = Label(root, text='아이디 ', width=10)
id_lbl.grid(row=0, column=0)
id_ent = Entry(root, width=20)
id_ent.grid(row=0, column=1)

pw_lbl = Label(root, text='비밀번호', width=10)
pw_lbl.grid(row=1, column=0)
pw_ent = Entry(root, width=20)
pw_ent.grid(row=1, column=1)

btn = Button(root, text='로그인', width=5, height=3)
btn.grid(row=0, column=2, rowspan=2)

root.mainloop()
```

실 행 결 과


Login
아이디
비밀번호
로그인
Copyright © Gilbut, Inc. All rights reserved.

위젯 5개를 만들고 `grid()`로 배치합니다. 첫 번째 행의 텍스트 레이블과 엔트리는 각각 `id_lbl`, `id_ent`로, 두 번째 행의 텍스트 레이블과 엔트리는 각각 `pw_lbl`, `pw_ent`로 만듭니다. [로그인] 버튼인 `btn`은 두 행을 병합해야 하므로 `rowspan`을 2로 설정합니다.

다음으로 기능 부분을 구현해 봅시다. [로그인] 버튼을 누르면 입력된 아이디와 비밀번호를 검증해 로그인 성공 또는 로그인 실패를 안내해야 합니다. 사용자 정의 함수 `login()`을 만들고 `btn` 버튼에 `command`로 연결합니다. 보통은 회원이 여러 명이지만, 여기에서는 간단하게 한 명의 회원만 로그인할 수 있게 합니다. 아이디 Lian, 비밀번호 1234로만 로그인되고 그밖의 조합은 로그인 실패로 처리합시다. 로그인 결과는 세 번째 행에 새로운 텍스트 레이블을 만들어 알려 주겠습니다. 이 코드도 먼저 작성해 본 후 책을 참고하세요. 앞 코드와 동일한 내용은 생략하고 추가되거나 달라진 부분만 나타냈습니다.

```
# ------ 프로그램 기능 부분 -----
def login():
    if id_ent.get() == 'Lian' and pw_ent.get() == '1234':
        result.set('로그인 성공!')
    else:
        result.set('로그인 실패. \n아이디와 비밀번호를 다시 확인하세요!')

# ----- 프로그램 화면 부분 -----
…
btn = Button(root, text='로그인', width=5, height=3, command=login)
btn.grid(row=0, column=2, rowspan=2)

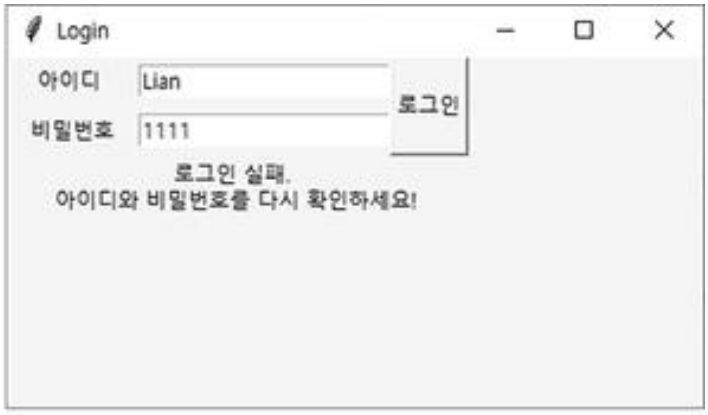
result = StringVar() # 로그인 성공 여부를 저장하는 동적변수
# 로그인 성공 여부를 나타내는 텍스트 레이블
result_lbl = Label(root, textvariable=result, width=30)
result_lbl.grid(row=2, column=0, columnspan=3)

root.mainloop()
```

실행결과



아이디와 비밀번호를 제대로 입력했을 때



아이디와 비밀번호를 잘못 입력했을 때

Copyright © Gilbut, Inc. All rights reserved.

동적변수 `result`를 정의하고 세 번째 행에 로그인 결과를 나타내는 `result_lbl` 레이블을 생성한 뒤 연결했습니다. `result_lbl` 레이블은 처음에 아무것도 없다가 로그인 버튼을 클릭하면 결과를 표시해야 하므로 변하는 값을 저장하는 동적 변수를 사용했습니다.

로그인 결과는 [로그인] 버튼을 눌렀을 때 결정됩니다. 로그인 성공 또는 로그인 실패 둘 중 하나죠. [로그인] 버튼을 눌렀을 때 `login()` 함수가 실행됩니다. 입력된 아이디와 비밀번호는 `id_ent.get()`과 `pw_ent.get()`으로 읽어옵니다. 조건문을 이용해 읽어온 아이디와 비밀번호가 Lian, 1234와 같으면 `result`에 '로그인 성공!'을 저장합니다. 아이디와 비밀번호 중 하나라도 다르면 '로그인 실패'를 저장합니다.

다양한 조합의 아이디와 비밀번호를 입력해 보세요. 입력값을 읽어서 조건문으로 판별한 후 결과를 화면에 나타내는 프로세스를 잘 알아두면 '사용자 - 화면 - 코드'의 상호작용을 이해하기 쉽습니다.

15.2.5 콤보박스

이번에는 콤보박스를 만들어 보겠습니다. 여러 선택지 중 일부를 선택하는 위젯은 콤보박스 외에도 라디오버튼, 체크박스 등이 있지만 이 책에서는 콤보박스만 다루겠습니다.

콤보박스 만들기

콤보박스(Combobox)는 클릭했을 때 여러 선택지가 드롭다운 형태로 나타나는 위젯입니다. 한 번에 여러 개를 보여줘야 하기 때문에 리스트로 데이터를 설정합니다. 콤보박스를 사용하려면 `from tkinter.ttk import *`를 추가해야 합니다. `ttk`는 `tkinter` 패키지 내부에 있는 모듈인데, `from tkinter import *`로는 추가되지 않아서 `from tkinter.ttk import *`를 추가로 작성해야 합니다. `ttk`는 콤보박스를 만드는 `Combobox` 클래스를 포함하고 있습니다.

`Combobox` 클래스로 콤보박스 객체를 생성하고, `values` 옵션으로 선택지를 담은 리스트를 설정합니다. 콤보박스를 만들어 봅시다.

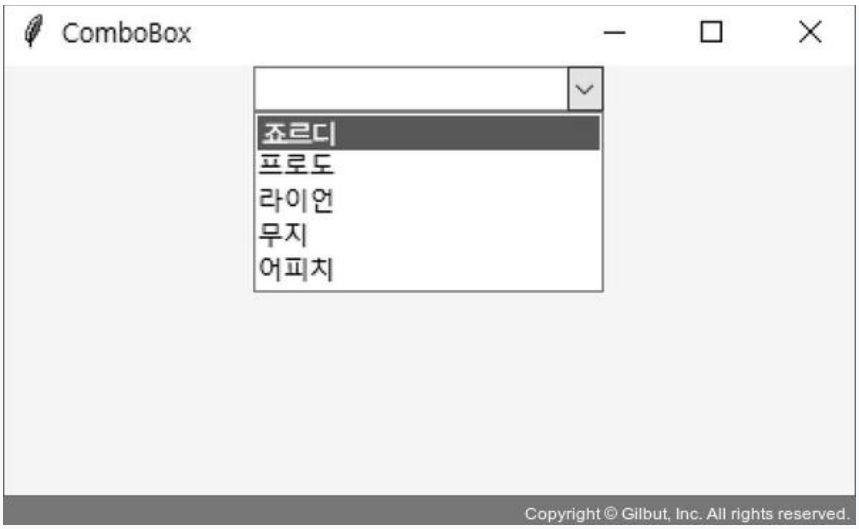
```
from tkinter import *
from tkinter.ttk import *

root = Tk()
root.title('ComboBox')
root.geometry('400x200')

# 콤보박스에 나타낼 리스트
characters = ['죠르디', '프로도', '라이언', '무지', '어피치']
# 콤보박스 객체 생성, characters 리스트 연결
combo = Combobox(root, values=characters)
combo.pack()

root.mainloop()
```

실행결과


ComboBox
죠르디
프로도
라이언
무지
어피치
Copyright © Gilbut, Inc. All rights reserved.

캐릭터 이름 5개가 들어간 `characters` 리스트를 만듭니다. 그리고 `Combobox` 클래스의 객체 `combo`를 만들면서 `values` 옵션에 `characters` 리스트를 값으로 설정합니다. 생성된 콤보박스를 클릭하면 리스트의 요소가 드롭다운 형태로 나타납니다.

이번에는 `grid()`를 사용해 콤보박스 왼쪽에 '캐릭터 이름'이라는 텍스트 레이블을, 오른쪽에 [선택]이라는 버튼을 추가해 봅시다. 버튼을 누르면 선택된 캐릭터의 이름을 출력합니다. 코드를 작성해 봅시다.

```
root = Tk()
root.title('ComboBox')
root.geometry('400x200')

def select():
    print('선택된 캐릭터는', combo.get())

lbl = Label(root, text='캐릭터 이름', width=10, anchor='center')
lbl.grid(row=0, column=0)

characters = ['죠르디', '프로도', '라이언', '무지', '어피치']
combo = Combobox(root, values=characters, width=30)
combo.grid(row=0, column=1)

btn = Button(root, text='선택', command=select, width=10)
btn.grid(row=0, column=2)

root.mainloop()
```

`실행결과`



실행 시



선택된 캐릭터는 어피치
어피치 선택 시

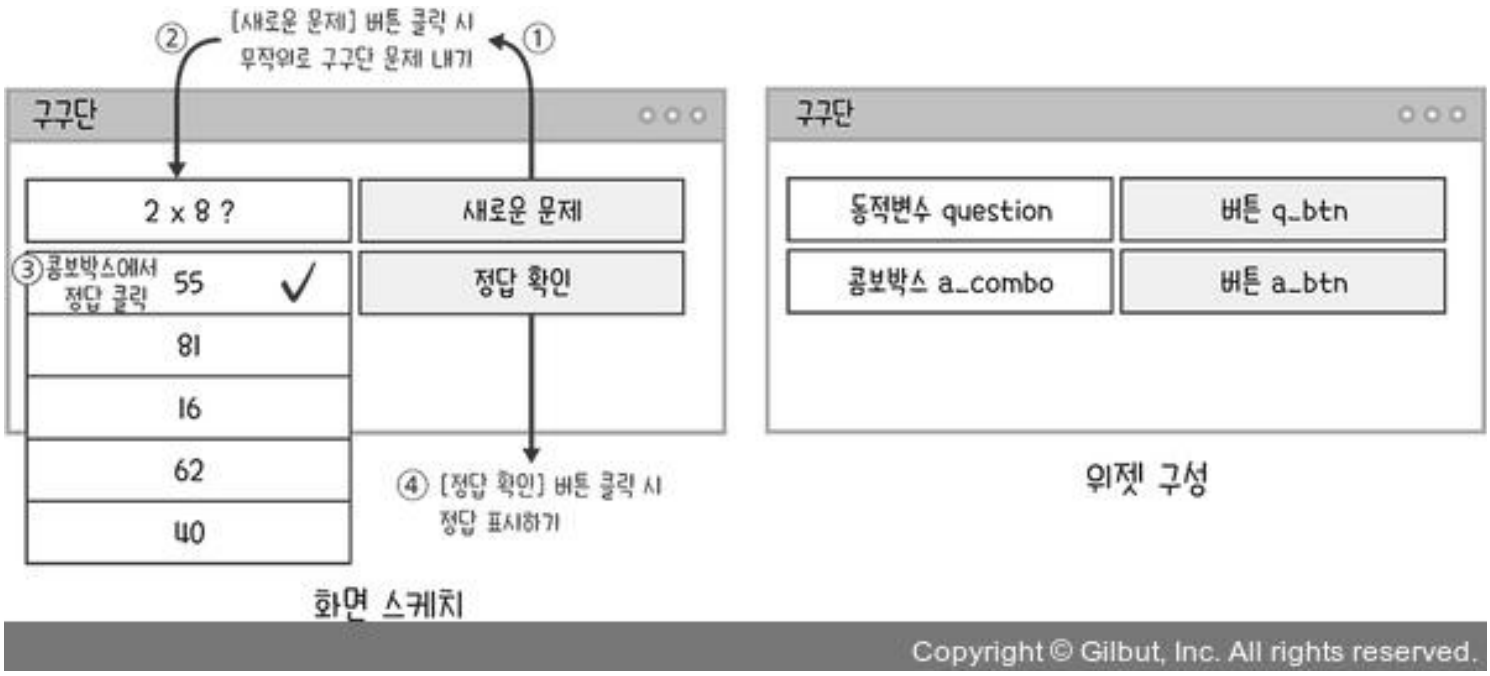
Copyright © Gilbut, Inc. All rights reserved.

화면 코드부터 봅시다. 레이블(`lbl`)과 버튼(`btn`)을 만들고 `grid()`로 콤보박스와 같은 행에 배치했습니다. 그리고 버튼을 클릭하면 `select()`가 실행되도록 `command`로 설정했습니다.

`select()` 함수 부분을 봅시다. 콤보박스에서 선택된 캐릭터 이름을 가져와 출력합니다. 콤보박스의 값은 엔트리처럼 `get()`으로 가져옵니다. 콤보박스에서도 화면에 값을 표시할 때는 `set()`을, 화면에서 값을 가져올 때는 `get()`을 주로 활용합니다.

콤보박스로 구구단 프로그램의 화면 만들기

콤보박스를 사용해 구구단 프로그램의 화면을 바꿔 봅시다. 이번에는 구구단 문제를 내면 콤보박스에서 5개의 선택지를 보여주고 답을 고르도록 프로그램을 작성합니다. 이때 콤보박스의 선택지 중 하나는 정답이고 나머지는 오답입니다. 정답을 선택한 후 [정답 확인] 버튼을 클릭하면 정답과 오답을 안내하겠습니다. 화면은 다음과 같이 구성합니다.



▲ 그림 15-12 변경된 구구단 프로그램 화면

프로그램 화면부터 작성해 봅시다. 앞에서 작성한 코드에서 정답을 출력하는 레이블 부분을 콤보박스로 변경하면 됩니다. 콤보박스에 표시할 선택지 리스트의 이름은 `selection`으로 하겠습니다.

```
from tkinter import *
from tkinter.ttk import *

# ----- 프로그램 화면 부분 -----
root = Tk()
root.title('구구단')
root.geometry('400x200')

question = StringVar()
q_lbl = Label(root, textvariable= question, width=30, anchor='center')
q_lbl.grid(row=0, column=0)
q_btn = Button(root, text='새로운 문제', width=10)
q_btn.grid(row=0, column=1)

selection = [] # 콤보박스에 연결할 리스트
a_combo = Combobox(root, values=selection, width=30)
a_combo.grid(row=1, column=0)
a_btn = Button(root, text='정답 확인', width=10)
a_btn.grid(row=1, column=1)

root.mainloop()
```

실행결과



앞에서 작성한 구구단 프로그램에서 창 크기를 추가하고, 정답을 보여 주는 레이블 대신 콤보박스를 추가했습니다.

버튼을 클릭할 때 실행되는 새로운 이벤트를 작성해 보겠습니다. [새로운 문제] 버튼을 클릭하면 `make_question()` 함수가 실행됩니다. 이때 구구단 문제를 무작위로 만들면서 동시에 콤보박스에 넣을 5개의 숫자도 만들어야 합니다. `selection`에는 정답 1개와 오답 4개가 필요합니다. 그리고 정답이 항상 같은 위치에 있으면 안 되겠죠? 정답 위치도 무작위로 결정돼야 합니다.

문제를 만드는 부분은 기존 코드를 그대로 사용하고 `selection` 부분만 추가하면 됩니다. [새로운 문제] 버튼을 클릭할 때마다 선택지 `selection`을 초기화해야 합니다. 이전 문제의 선택지를 비워야 하기 때문이죠. 그리고 오답으로 제시할 무작위 숫자 4개를 `selection`에 추가해야 하고요. 오답은 구구단 결괏값 중 가장 작은 숫자인 2부터 가장 큰 숫자인 81까지 중에서 4개를 무작위로 넣겠습니다.

```
import random

# ----- 프로그램 기능 부분 -----
num1, num2 = 0, 0

def make_question():
    global num1, num2
    # 문제 만들기
    num1, num2 = random.randint(2, 9), random.randint(1, 9)
    question.set(str(num1) + 'x' + str(num2) + '? ')
    # 선택지 초기화하고 새로 만들기
    selection = []
    for i in range(4):
        selection.append(random.randint(2, 81))
```

정답을 끼워 넣을 차례입니다. 현재 `selection`에는 4개의 요소가 있으므로 인덱스 [0]부터 [3]까지 있습니다. 정답을 추가하면 인덱스는 [4]까지로 늘겠죠. 정답은 `selection`의 무작위 위치에 들어가야 하므로 0~4 중 하나를 무작위로 선택해야 합니다. 6장에서 다루지는 않았지만 리스트 중간에도 값을 추가할 수 있습니다. 형식은 다음과 같습니다.

형식

```
리스트명.insert(인덱스, 값)
```

`randint()`를 활용해 0부터 4 사이에서 인덱스를 구하고, 해당 인덱스에 정답인 `num1 * num2`를 끼워 넣으면 됩니다. 다음으로 완성된 선택지 `selection`을 콤보박스와 다시 연결해야겠죠? 콤보박스의 `values`에 새로운 값은 `콤보박스명['vaules'] = 리스트명` 형식으로 넣습니다. 따라서 여기에서는 `a_combo['values'] = selection`이 됩니다. `set()`을 사용하면 `selection`의 각 요소가 모두 한 줄로 표시되어 선택지가 하나로 나오기 때문에 이 방식을 사용합니다.

```
import random

# ----- 프로그램 기능 부분 -----
num1, num2 = 0, 0

def make_question():
    global num1, num2
    # 문제 만들기
    num1, num2 = random.randint(2, 9), random.randint(1, 9)
    question.set(str(num1) + 'x' + str(num2) + '? ')
    # 선택지 초기화하고 새로 만들기
    selection = []
    for i in range(4):
        selection.append(random.randint(2, 81))
    # 무작위 위치에 정답 끼워 넣기
    idx = random.randint(0, 4)
    selection.insert(idx, num1 * num2)

    # 콤보박스에 변경된 리스트 연결하기
    a_combo['values'] = selection
```

[새로운 문제] 버튼(`q_btn`)에 완성된 `make_question` 함수를 이벤트로 연결한 후 프로그램을 실행해 봅시다. 프로그램 기능 부분과 화면 부분 코드를 합쳐서 실행해야 합니다. 통상적으로 함수를 위에 작성하니 프로그램 기능 → 프로그램 화면 순서로 작성하는 것이 좋습니다.

```
q_btn = Button(root, text='새로운 문제', width=10, command=make_question)
```

실행결과



[새로운 문제] 버튼을 클릭할 때마다 다른 구구단 문제가 나타나고, 콤보박스에도 새로운 선택지가 나옵니다.

15.2.6 메시지박스

메시지박스(Messagebox)는 웹에서 흔히 볼 수 있는 알림창입니다. 메시지박스는 메시지 종류에 따라 기본, 정보, 경고, 질문 유형이 있는데, 여기서는 정보와 오류 유형의 메시지박스를 사용하겠습니다. 다른 유형은 https://docs.python.org/3/library/tkinter.messagebox.html?highlight=messagebox를 참고해 주세요.

메시지박스 만들기

메시지박스를 사용하려면 `tkinter` 패키지의 `messagebox` 모듈을 포함해야 합니다. 콤보박스 때처럼 `from tkinter.messagebox import *`로 작성하면 됩니다. 이렇게 모듈을 포함한 후 `help(messagebox)`를 실행하면 메시지박스에 대한 정보를 확인할 수 있습니다.

정보 유형의 메시지박스는 `showinfo()` 함수로 표시합니다. `showinfo()`는 다음과 같이 사용합니다.

형식

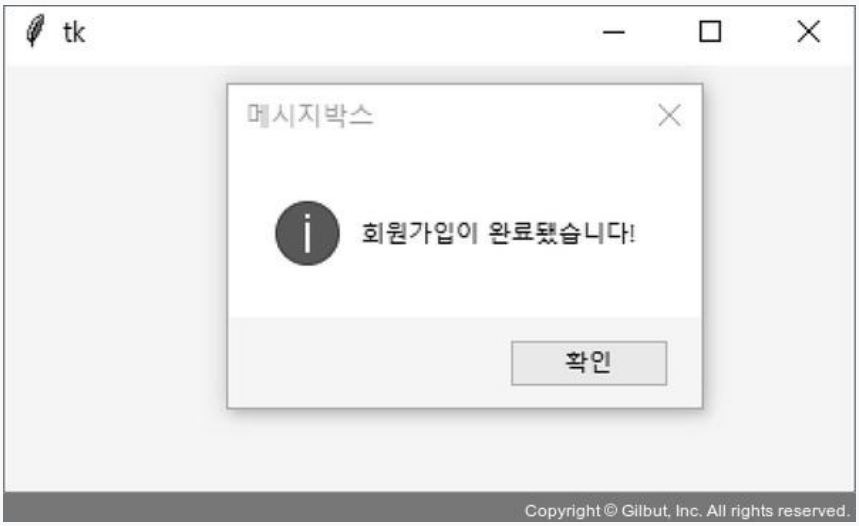
```
showinfo(title='팝업창명', message='메시지')
```

다음 코드를 실행해 봅시다.

```
from tkinter import *
from tkinter.messagebox import *

root = Tk()
root.geometry('400x200')
showinfo(title='메시지박스', message ='회원가입이 완료됐습니다!')
root.mainloop()
```

실행결과

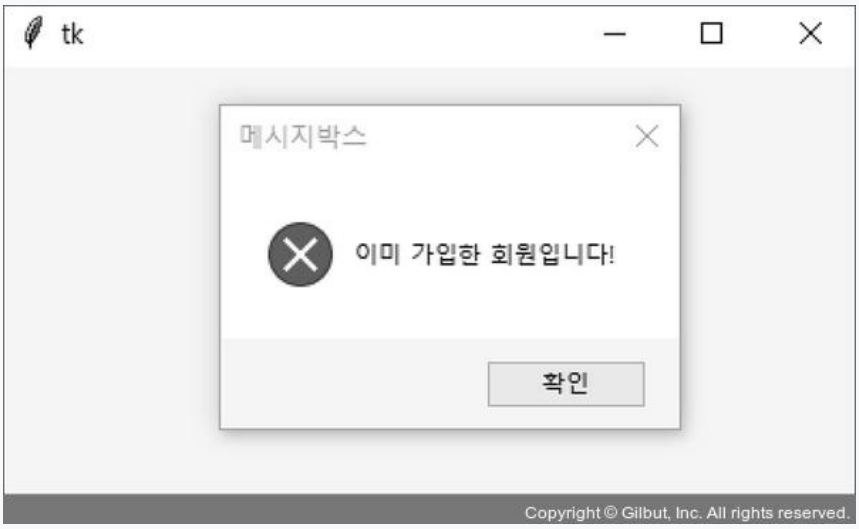


`showinfo()`는 information의 약자인 i가 이미지로 나타납니다.

오류 유형의 메시지 박스는 `showerror()`를 사용합니다. 사용 방식은 동일합니다. 앞의 코드에서 `showinfo()` 부분만 변경한 후 실행해 봅시다.

```
showerror(title='메시지박스', message='이미 가입한 회원입니다!')
```

실행결과



`showerror()`는 X 표시가 이미지로 나타납니다.

구구단 프로그램에 메시지박스 추가하기

구구단 프로그램에서 정답을 확인하는 부분을 아직 작성하지 않았습니다. 이번에는 선택한 답이 맞는지 확인하는 `check_answer()` 함수를 작성한 후 [정답 확인] 버튼을 클릭하면 선택한 콤보박스의 값이 정답인지 아닌지를 알려 주는 메시지박스를 추가해 보겠습니다.

앞의 구구단 프로그램에서 [정답 확인] 버튼을 클릭했을 때 `check_answer()` 함수가 실행되게 합니다. `check_answer()` 함수에서는 선택한 값이 정답이면 '정답입니다!'를, 정답이 아닐 때는 '다시 한번 풀어 보세요.'를 메시지박스로 표시합니다. 전체 코드를 작성해 봅시다.

```
from tkinter import *
from tkinter.ttk import *
from tkinter.messagebox import *
import random

# ----- 프로그램 기능 부분 -----
# 전역변수
num1, num2 = 0, 0
selection = [] # 전역변수로 위치 이동

def make_question():
    global num1, num2
    # 문제 만들기
    num1, num2 = random.randint(2, 9), random.randint(1, 9)
    question.set(str(num1) + 'x' + str(num2) + '? ')

    # 선택지 만들기
    for i in range(4):
        selection.append(random.randint(2, 81))
    idx = random.randint(0, 4)
    selection.insert(idx, num1 * num2)
    a_combo['values'] = selection

def check_answer():
    if int(a_combo.get()) == num1 * num2:
        showinfo(title='정답 확인!', message='정답입니다.')
    else:
        showerror(title='정답 확인!', message='다시 한번 풀어 보세요!')

# ----- 프로그램 화면 부분 -----
root = Tk()
root.title('구구단')
root.geometry('400x200')
question = StringVar()
q_lbl = Label(root, textvariable=question, width=30, anchor='center')
q_lbl.grid(row=0, column=0)
q_btn = Button(root, text='새로운 문제', width=10, command=make_question)

q_btn.grid(row=0, column=1)
a_combo = Combobox(root, values=selection, width=30)
a_combo.grid(row=1, column=0)
a_btn = Button(root, text='정답 확인', width=10, command=check_answer)
```

```
a_btn.grid(row=1, column=1)


root.mainloop()
```

실행결과



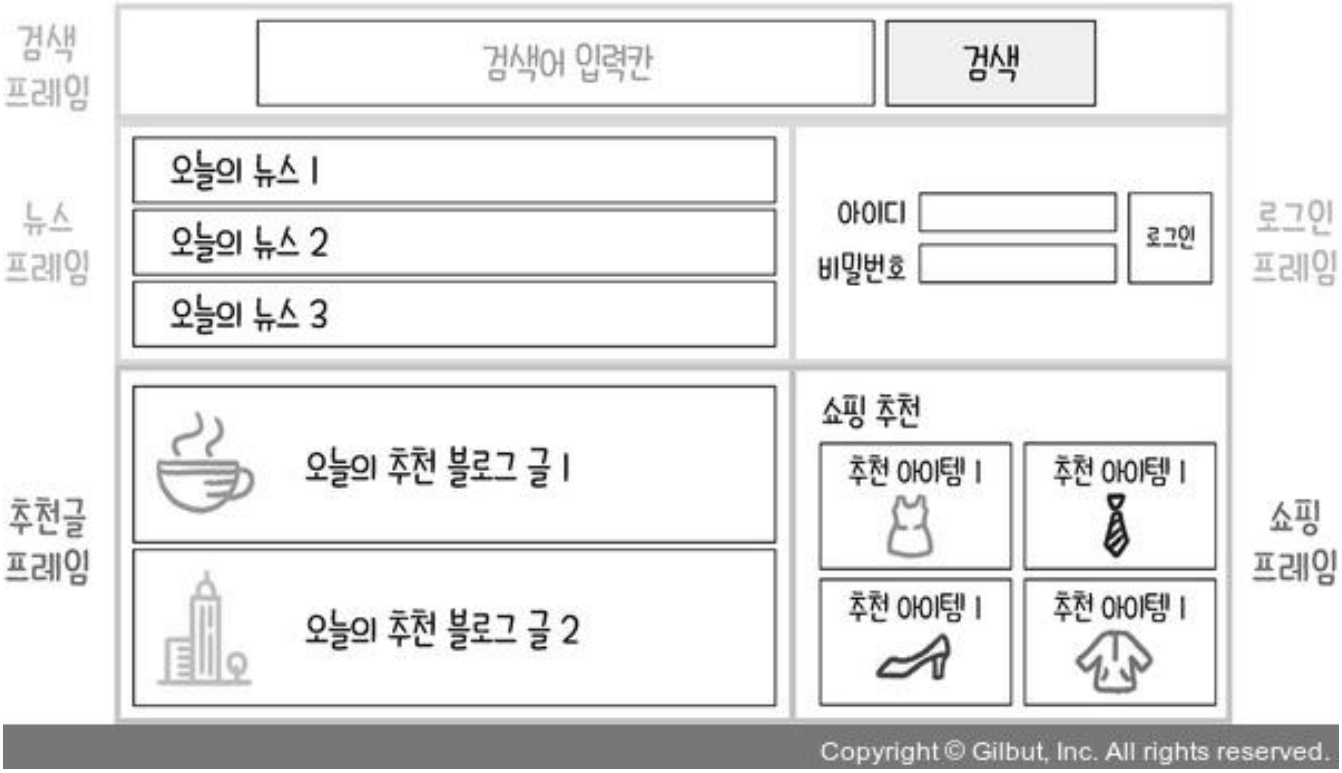
[정답 확인] 버튼(`a_btn`)에 `command`로 `check_answer()` 함수를 연결했습니다. 기존의 `check_answer()` 함수는 정답을 계산해 동적변수 `answer`에 넣는 함수였죠. 여기서는 콤보박스의 값을 읽어와 정답과 비교하는 함수로 바꿨습니다. 콤보박스의 값은 `a_combo.get()`으로 읽어오는데, 이때 유의할 점이 있습니다. 읽어온 값은 문자열로 인식되기 때문에 정수형 정답(`num1 * num2`)과 비교하려면 `int()`로 감싸서 정수형으로 바꿔야 합니다. 그리고 이 값을 정답과 비교해 일치하는 경우에는 `showinfo()`를, 일치하지 않는 경우에는 `showerror()`를 실행하도록 조건문을 작성했습니다. 함수 내부에 있던 `selection = []`는 프로그램 전체에서 사용하므로 전역변수 정의 부분으로 올렸습니다. 정답도 선택하고 오답도 선택하며 메시지박스가 잘 뜨는지 확인해 보세요.

이제 그럴듯한 구구단 프로그램이 됐습니다. 일부 위젯만 알아봤지만, '사용자 - 화면 - 코드' 간의 흐름을 이해한다면 다른 위젯도 사용하기 어렵지 않을 겁니다.

15.2.7 프레임

웹 페이지를 보면 화면을 구역별로 나눠 비슷한 콘텐츠끼리 묶어 놓는 것을 볼 수 있습니다. 이렇게 웹 페이지 화면을 구역별로 나누는 것을 프레임(frame)이라고 합니다.

예를 들어, 우리에게 익숙한 포털 사이트 화면은 다음 그림처럼 구역을 나눌 수 있습니다.



▲ 그림 15-13 웹 사이트 화면의 프레임

`tkinter`에서도 프레임 위젯을 활용해 비슷한 성격을 가진 위젯이나 위치가 가까운 위젯끼리 묶어 구역을 설정할 수 있습니다. 다른 위젯과 마찬가지로 `Frame` 클래스의 객체를 만들고, 이 객체에 위젯을 배치하면 해당 프레임에 위젯이 나타납니다. 프레임도 여러 개를 만들어 배치할 수 있습니다.

로그인 화면에 프레임 만들기
엔트리에서 만든 로그인 화면을 수정해 2개의 프레임으로 구성해 보겠습니다.

```
from tkinter import *

# ----- 프로그램 화면 부분 -----
root = Tk()
root.title('frame')
root.geometry('400x200')
frame_A = Frame(root, relief='groove', borderwidth=2)
frame_A.grid(row=0, column=0)
frame_B = Frame(root, relief='groove', borderwidth=2)
frame_B.grid(row=1, column=0)

id_lbl = Label(frame_A, text='아이디 ', width=10, height=2)
id_lbl.grid(row=0, column=0)
id_ent = Entry(frame_A, width=20)
id_ent.grid(row=0, column=1)

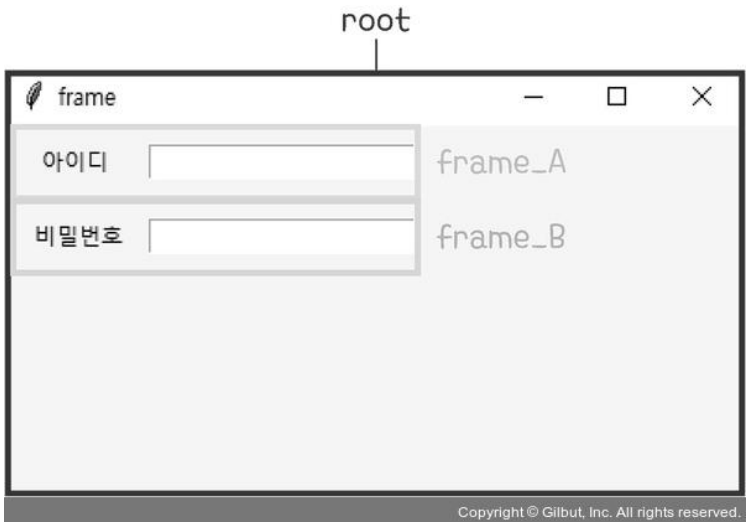
pw_lbl = Label(frame_B, text='비밀번호', width=10, height=2)
pw_lbl.grid(row=0, column=0)
pw_ent = Entry(frame_B, width=20)
pw_ent.grid(row=0, column=1)

root.mainloop()
```

실행결과


frame
아이디
비밀번호
Copyright © Gilbut, Inc. All rights reserved.

기본 창인 `root`를 먼저 만들었듯이 위젯을 만들기 전에 위젯을 배치할 프레임을 먼저 만들어야 합니다. 여기서는 `frame_A`와 `frame_B`라는 2개의 프레임 객체를 만들었습니다. 이렇게 만든 프레임도 기존 창에 구역을 만드는 것이므로 `root`를 기본 창으로 설정했습니다. 그리고 프레임을 확실하게 표현하기 위해 테두리 모양은 `relief='groove'`로, 테두리 두께는 `borderwidth=2`로 설정했습니다. `frame_B`도 같은 방법으로 만듭니다. 프레임 역시 `place()`, `pack()`, `grid()`로 배치할 수 있는데, 여기서는 `grid()`를 사용해 `frame_A`를 첫 번째 행에, `frame_B`를 두 번째 행에 배치했습니다.



▲ 그림 15-14 프레임의 화면 배치

프레임 생성과 배치가 끝나면 위젯을 배치합니다. 아이디를 출력하는 레이블 객체 `id_lbl`과 아이디를 입력받는 엔트리 객체 `id_ent`를 만들 때, 위치할 곳을 `root`가 아닌 `frame_A`로 지정합니다. 그래야 `frame_A` 구역에 두 위젯이 배치됩니다. `pw_lbl`과 `pw_ent`도 마찬가지로 위치할 곳을 `frame_B`로 지정합니다. 위젯을 배치할 때는 전체 창(`root`)이 아닌, 해당 프레임 내에서의 위치를 지정하면 됩니다. 예를 들어 `pw_lbl`과 `pw_ent`는 `root`를 기준으로 하면 `row=1` 위치에 있지만, `frame_B`를 기준으로 했을 때 `row=0`에 위치합니다. 따라서 `row=0`으로 설정합니다.

프레임을 설정한 웹 페이지 화면 만들기

이번에는 포털 사이트의 메인 화면을 단순하게 만들어 보겠습니다. 가장 위에 사이트 로고와 이름이 있고, 그 아래에는 정보를 검색할 수 있도록 검색 구역과 오늘의 날씨를 알려 주는 날씨 구역을 넣어 봅시다.

```
from tkinter import *

root = Tk()
root.title('렛츠기릿 포털 사이트')
root.geometry('400x200')

# ----- 프레임 생성 -----
frame_logo = Frame(root)
frame_logo.grid(row=0, column=0)
frame_search = Frame(root)
frame_search.grid(row=1, column=0)
frame_weather = Frame(root)
frame_weather.grid(row=2, column=0)

# ----- 로고 프레임 -----
logo = PhotoImage(file='lets get it.png')
logo = logo.subsample(6, 6)
logo_label = Label(frame_logo, image=logo)
logo_label.grid(row=0, column=0)
site_title = Label(frame_logo, text='렛츠기릿', font=('Malgun Gothic', 20))
site_title.grid(row=0, column=1)

# ----- 검색 프레임 -----
search_lbl = Label(frame_search, text='검색어 입력', width=10)
search_lbl.grid(row=0, column=0)
search_ent = Entry(frame_search, width = 35)
search_ent.grid(row=0, column=1)
search_btn = Button(frame_search, text='검색', width=8)
search_btn.grid(row=0, column=2)

# ----- 날씨 프레임 -----
weather_title = Label(frame_weather, text='오늘의 날씨', font=('Malgun Gothic', 10), width=10)
weather_title.grid(row=0, column=0, rowspan =2)
weather_temperature = Label(frame_weather, text='28도', font=('Malgun Gothic', 14), width=5)
weather_temperature.grid(row=0, column=1, rowspan=2)
weather_explain = Label(frame_weather, text='흐림, 어제보다 0도 높아요. 체감온도 30.4', width=3
5)
weather_explain.grid(row=0, column=2)
weather_info = Label(frame_weather, text='미세먼지 좋음 / 초미세먼지 좋음', width=35)
weather_info.grid(row=1, column=2)
```

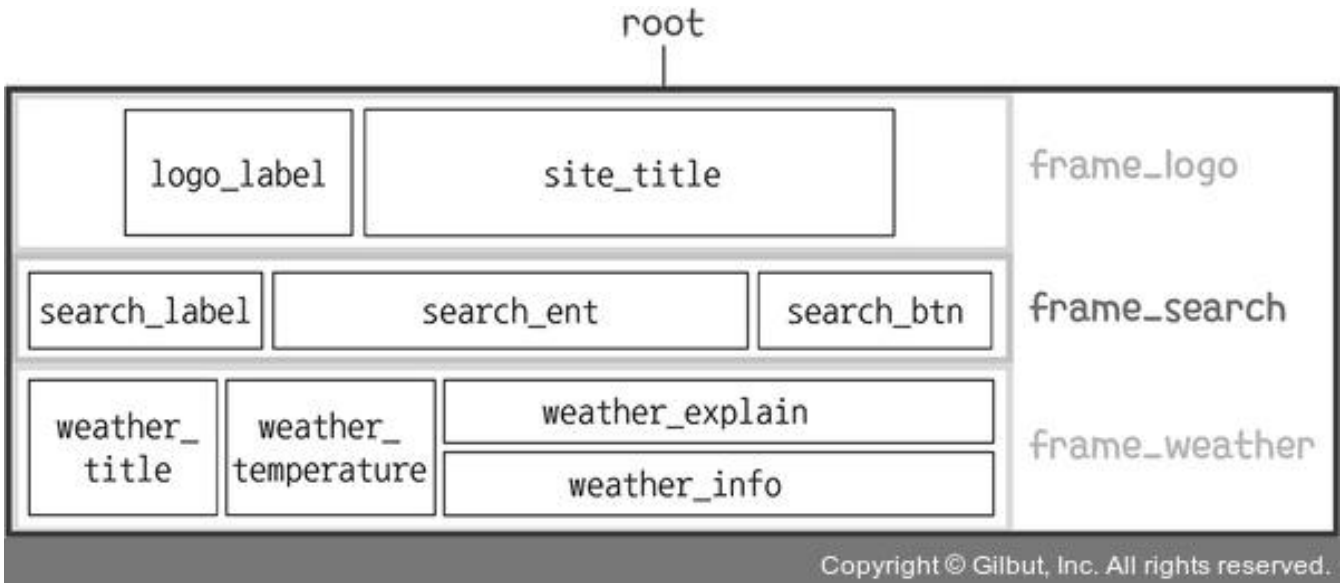
```
root.mainloop()
```

실행결과



총 3개의 프레임을 만들고 프레임에 각각 위젯을 넣었습니다. `frame_logo`, `frame_search`, `frame_weather`를 위에서부터 순서대로 배치하고 `frame_logo`에는 텍스트 레이블과 이미지 레이블로 사이트 이름과 로고를, `frame_search`에는 레이블, 엔트리, 버튼으로 '검색어 입력' 텍스트, 검색창, 검색 버튼을 일렬로 배치했습니다. `frame_weather`에는 레이블로 '오늘의 날씨' 텍스트, 기온과 날씨 상세 내용을 배치했습니다.

코드의 전체 구조는 다음과 같습니다.



▲ 그림 15-15 프레임과 위젯의 구조

프레임을 사용하지 않고 앞의 위젯을 전부 `root`에 `grid()`로 배치한다면 격자의 칸별로 너비와 높이를 통일해야 하기 때문에 유연하게 배치하기가 어렵습니다. 그리고 추가하거나 수정할 사항이 생겼을 때 위젯의 배치를 모두 수정해야 할 수도 있어서 매우 비효율적입니다.

유사한 기능이나 인접한 위젯끼리 프레임으로 묶으면 해당 프레임 안에서는 다른 프레임과 다른 새로운 배치를 사용할 수도 있고, 위젯을 추가 또는 삭제하더라도 해당 프레임 안에서만 수정하면 되기 때문에 프로그램의 유지 보수가 훨씬 쉽습니다.

1분 퀴즈

2 다음은 `sayhello` 리스트의 값을 화면에 보여 주는 프로그램입니다. [-->] 버튼을 누를 때마다 화면에 현재 보이는 값의 오른쪽 요소 값이 화면에 표시됩니다. 코드를 보고 틀린 설명을 고르세요.

```
from tkinter import *

sayhello = ['안녕하세요', 'hello', '니하오', '봉쥬르']
idx = 0    # sayhello 리스트의 인덱스

# ----- 프로그램 기능 부분 -----
def plus():
    (가)
    idx = (idx + 1) % 4
    (나)

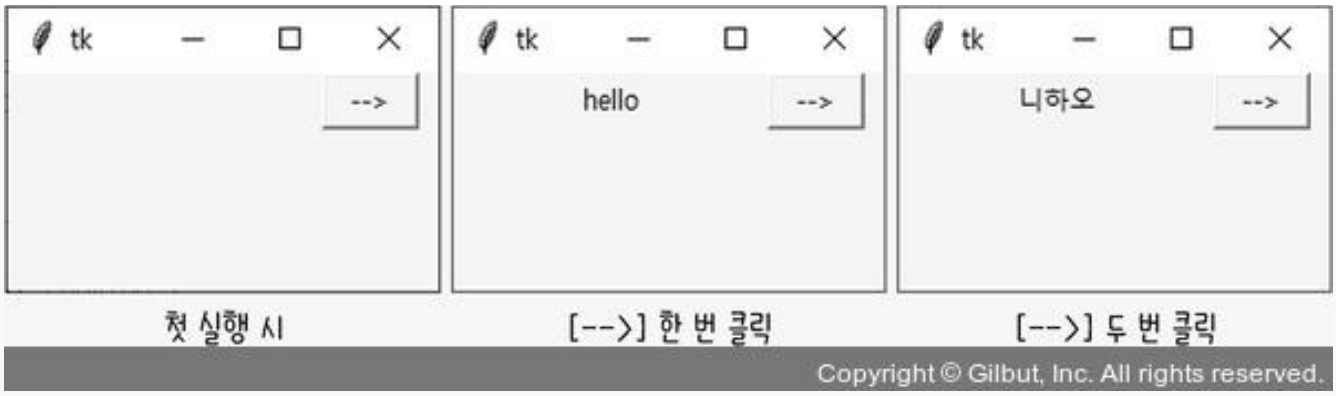
# ----- 프로그램 화면 부분 -----
root = Tk()
root.geometry('200x100')

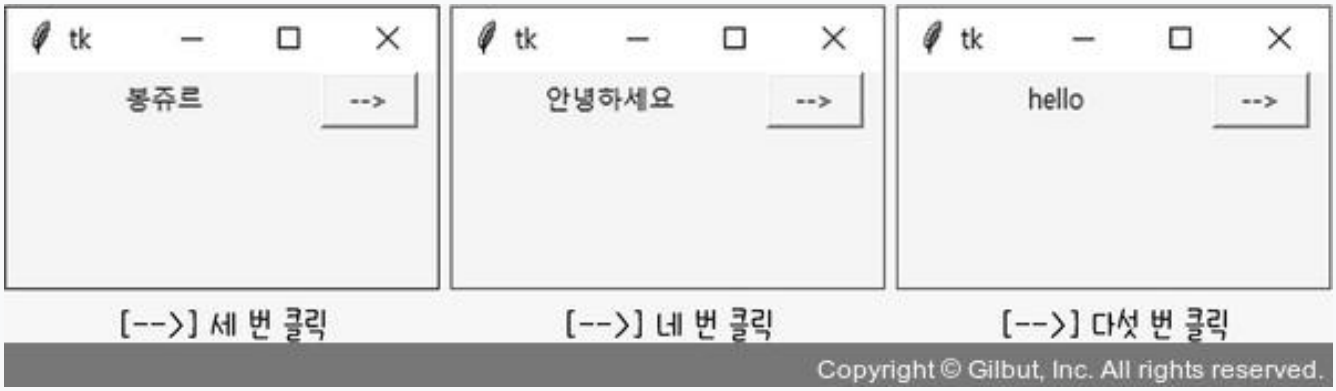
hello_txt = StringVar()
hello_lbl = Label(root, textvariable=hello_txt, width=20)
hello_lbl.grid(row=0, column=0)

btn_plus = Button(root, text='-->', command=plus, width=5)
btn_plus.grid(row=0, column=1)

root.mainloop()
```

실행결과





① `idx`는 전역변수다.

② `plus()` 함수에서 `idx`를 변경하려면 (가)에는 `global idx`가 들어가야 한다.

③ (나)에 들어갈 코드는 `hello_txt.get(sayhello[idx])`다.

④ `hello_txt`는 동적변수다.

⑤ 버튼을 클릭할 때마다 `plus()`가 실행된다.

정답 및 해설

15.3

핫플레이스 프로젝트에 화면 입히기

핫플레이스 프로젝트에 `tkinter`를 사용해 화면을 입혀 보겠습니다. 지금까지는 코드 결과창에 지역별 그래프를 그렸는데, 이번에는 코드를 숨기고 프로그램 화면으로만 표현해 결과가 더 직관적으로 보이는 프로그램을 만들어 봅시다.

기본 기능은 13장에서 완성했으므로 코드를 프로그램 화면에 맞게 수정하는 방향으로 작성해 보겠습니다. 13장에서 완성한 핫플레이스 프로젝트 파일을 열어 두고, 주피터 노트북에 새로운 파이썬 파일을 하나 만듭니다. 인구 데이터 파일(LOCAL_PEOPLE_DONG_201912.csv)과 행정동코드 파일(dong_code.csv), 위치 데이터 파일(dong_location.csv)도 새로 만든 파이썬 파일과 같은 폴더에 있어야 합니다.

15.3.1 프로그램 화면 만들기

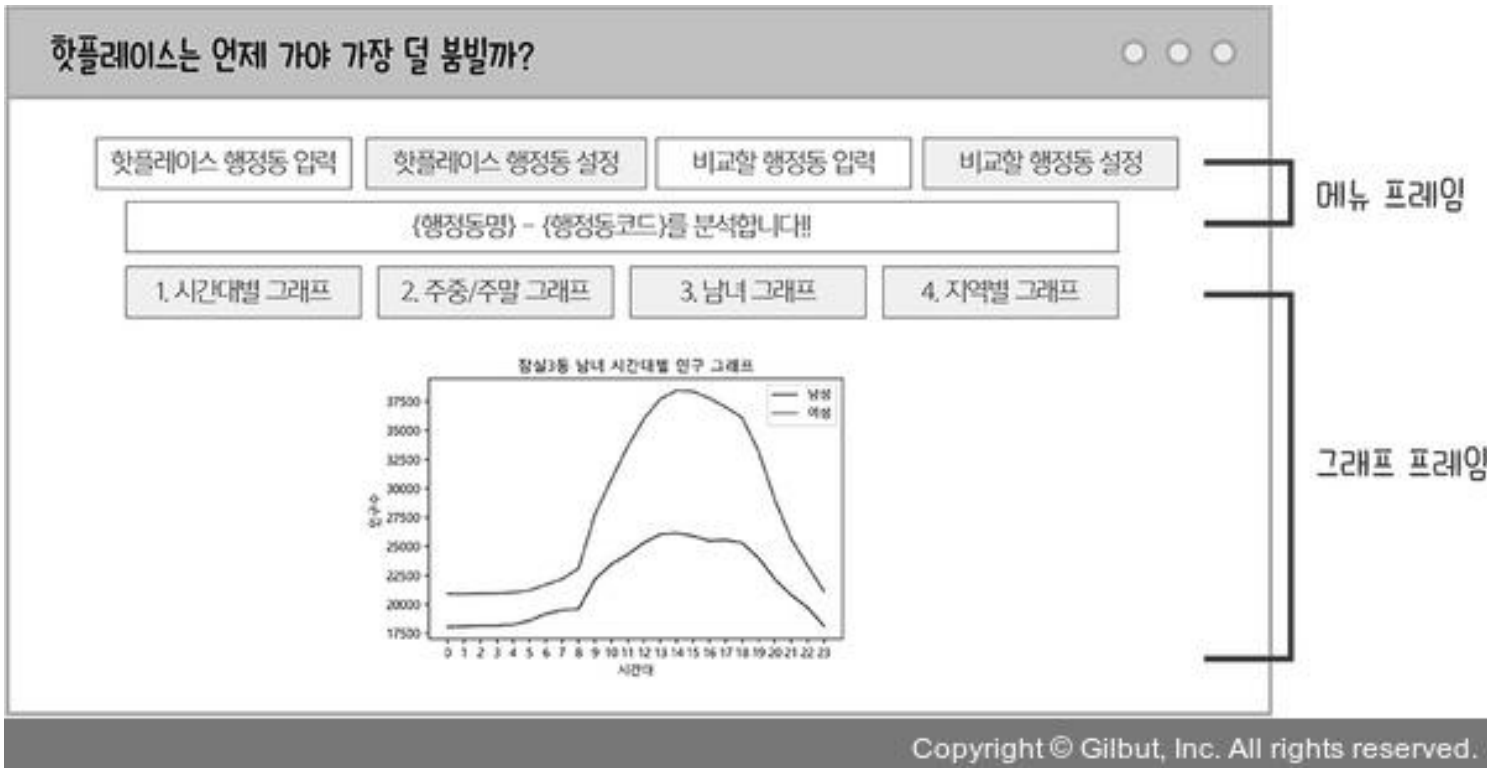
화면부터 구성해 봅시다. 핫플레이스 프로젝트에서 화면에 보여야 하는 부분은 어디일까요? 입력이 필요한 부분과 결과겠죠.

입력 부분부터 봅시다. 분석이 시작되려면 행정동명을 입력받아야 합니다. 그리고 입력받은 행정동명의 행정동코드를 찾아야죠. 입력은 엔트리로 받고, 행정동코드를 찾는 함수가 실행되도록 버튼으로 이벤트를 설정합니다. 버튼은 각 엔트리 옆에 배치합시다. 입력할 부분이 또 있습니다. 하위 목표 4에서 두 지역을 비교할 때도 비교할 행정동명을 입력받는 엔트리와 행정동코드로 변환하는 버튼이 필요합니다.

추가로 사용자가 보기 편하게, 분석을 시작하기 전에 인구 분석을 시작한다는 안내 문구를 넣겠습니다. 입력된 행정동명에 따라 출력되는 내용이 달라지므로 동적변수를 활용한 레이블을 만들면 됩니다.

프로젝트에서 진행하는 분석은 총 4가지입니다. 따라서 어떤 분석을 진행할지도 입력받아야 합니다. 이런 경우에는 버튼을 사용하면 좋겠죠? 분석별로 각각 버튼을 만듭시다. 마지막으로 버튼을 클릭하면 결과 분석 그래프가 화면에 보이게 합니다.

정리해서 화면을 스케치해 보겠습니다.



▲ 그림 15-16 핫플레이스 프로젝트 프로그램 화면

화면을 스케치하고 보니 구역이 보입니다. 프레임을 활용해 위젯을 묶을 수 있겠네요. 크게 메뉴 영역과 그래프 영역으로 나눠 프레임을 구성하고 개별 위젯을 만들어 보겠습니다.

코드를 작성해 봅시다.

```
from tkinter import *

# ----- 프로그램 화면 부분 -----
root = Tk()
root.geometry('800x500')
root.title('핫플레이스는 언제 가야 가장 덜 붐빌까?')

# ----- 프레임 -----
Menu = Frame(root)
Menu.grid(row=0, column=0)
Graph = Frame(root)
Graph.grid(row=1, column=0)

# ----- 메뉴 프레임 -----
dong_ent1 = Entry(Menu, width=20)
dong_ent1.grid(row=0, column=0)

dong_btn1 = Button(Menu, text='핫플레이스 행정동 설정', width=20)
dong_btn1.grid(row=0, column=1)

dong_ent2 = Entry(Menu, width=20)
dong_ent2.grid(row=0, column=2)

dong_btn2 = Button(Menu, text='비교할 행정동 설정', width=20)
dong_btn2.grid(row=0, column=3)

info = StringVar()
dong_lbl = Label(Menu, textvariable=info, font=('Malgun Gothic', 12))
dong_lbl.grid(row=1, columnspan=4)

# ----- 그래프 프레임 -----
btn1 = Button(Graph, text='1. 시간대별 그래프', width=20)
btn1.grid(row=0, column=0)

btn2 = Button(Graph, text='2. 주중/주말 그래프', width=20)
btn2.grid(row=0, column=1)

btn3 = Button(Graph, text='3. 남녀 그래프', width=20)
btn3.grid(row=0, column=2)

btn4 = Button(Graph, text='4. 지역별 그래프', width=20)
btn4.grid(row=0, column=3)
```

```
graph_lbl = Label(Graph)
graph_lbl.grid(row=1, column=0, columnspan=5)

root.mainloop()
```

실 행 결 과



기본 창 `root`에 `Menu`와 `Graph`, 2개의 프레임을 배치했습니다. `Menu` 프레임에는 핫플레이스 행정동을 텍스트로 입력받는 `dong_ent1` 엔트리, 행정동코드로 변환하는 버튼의 `dong_btn1`, 비교할 행정동을 입력받는 `dong_ent2` 엔트리, 비교할 지역을 행정동코드로 변환하는 `dong_btn2`를 배치했습니다. `Menu`의 두 번째 행에는 '행정동명-행정동코드를 분석합니다!'가 출력되도록 동적변수 `info`를 정의하고 `dong_lbl` 레이블 객체를 만들었습니다.

15.3.2 프로그램 기능 구현하기

이번에는 13장 코드를 기반으로 프로그램 기능 부분을 수정해 봅시다..

파일 읽기

가장 먼저 인구 분석에 필요한 파일을 읽어야 하죠. 파일을 읽는 `file_open()` 함수를 화면에 맞게 수정해 보겠습니다. 화면 구현 부분과 프로그램 기능 구현 부분을 분리하기 위해(코드의 가독성이 좋아집니다) 에디터에서 `tkinter` 화면을 만드는 코드를 작성한 셀 위에 셀을 하나 추가하고 `file_open()`을 작성합니다. 13장에서 만든 함수와 같으니 그대로 복사해도 됩니다.

```
import csv

# ----- 프로그램 기능 부분 -----
def file_open():
    f = open('LOCAL_PEOPLE_DONG_201912.csv', encoding='utf8') # 또는 'eur-kr', 'cp949'
    data = csv.reader(f)
    next(data)
    data = list(data)

    f2 = open('dong_code.csv', encoding = 'cp949') # 또는 'eur-kr', 'utf8'
    code_data = csv.reader(f2)
    next(code_data)
    next(code_data)
    code_data = list(code_data)

    for row in data:
        for i in range(1, 32):
            if i <= 2:
                row[i] = int(row[i])
            else:
                row[i] = float(row[i])

    for row in code_data:
        row[1] = int(row[1])

    return data, code_data
```

화면이 만들어지기 전에 데이터를 읽어야 합니다. 화면에는 파일 읽기를 실행하는 버튼이 없기 때문입니다. 따라서 파일 읽기 함수와 화면 코드 사이에 파일을 읽고 전역변수를 정의하는 `data`, `code_data = file_open()`을 넣습니다.

```
# ----- 프로그램 기능 부분 -----
def file_open():
    ...

# 전역변수
data, code_data = file_open()

# ----- 프로그램 화면 부분 -----
root = Tk()
…
```

행정동명을 행정동코드로 변환하기
입력받은 행정동명의 행정동코드를 찾는 함수를 만들어 봅시다. 엔트리에 입력된 행정동명을 가져와 행정동코드 데이터인 `code_data`에서 검색해야 하죠? 비교 지역까지 2개의 행정동명을 입력받기 때문에 4개의 전역변수를 만들고 검색 함수도 2개를 만듭니다.

```
# ----- 프로그램 기능 부분 -----
def file_open():
    ...

# 핫플레이스가 있는 행정동명의 행정동코드(dong_code1)를 찾는 함수
def dong_search1():
    global dong_name1, dong_code1
    dong_name1 = dong_ent1.get()

    for row in code_data:
        if row[-1] == dong_name1:
            dong_code1 = row[1]
    info.set(dong_name1 + '-' + str(dong_code1) + '을(를) 분석합니다!')

# 비교할 행정동명의 행정동코드(dong_code2)를 찾는 함수
def dong_search2():
    global dong_name2, dong_code2
    dong_name2 = dong_ent2.get()

    for row in code_data:
        if row[-1] == dong_name2:
            dong_code2 = row[1]
    info.set(dong_name1 + '과 '+ dong_name2 + '을(를) 비교합니다!')

# 전역변수
data, code_data = file_open()
dong_name1, dong_code1 = '', ''   # 핫플레이스가 있는 행정동명과 행정동코드
dong_name2, dong_code2 = '', ''   # 비교할 행정동명과 행정동코드
```

`dong_name1`, `dong_name2`와 `dong_code1`, `dong_code2`를 전역변수로 정의해야 합니다. 함수에서 사용하는 전역변수는 함수 정의 전에 있는 것이 인지 순서상 더 편하지만, 코드의 가독성을 위해 기존 전역변수가 있는 곳 아래에 작성하겠습니다. 함수에서 사용하는 전역변수는 해당 함수 호출 전에만 정의되면 됩니다. 네 변수 모두 초깃값으로 빈 값을 넣어 줍니다.

`tkinter`에서 버튼의 이벤트로 연결되는 함수는 인자를 넣어서 전달할 수 없습니다. 예를 들어 `command=dong_search('합정동')`이 아닌 `command=dong_search`로 작성해야 합니다. 그래서 기존 `dong_search()` 함수를 `dong_search1()`, `dong_search2()`로 나누고 매개변수 없이 정의합니다. 각각 핫플레이스가 있는 행정동코드와 비교할 행정동의 코드를 찾는 함수입니다.

행정동명을 엔트리 입력에서 읽어와야 하므로 `dong_ent1.get()`, `dong_ent2.get()`으로 읽어와서 전역변수 `dong_name1`과 `dong_name2`에 값을 저장합니다. 이때 함수에서 전역변수의 값을 변경하므로 `global`로 선언합니다. 그리고 이 값을 `code_data`의 `row[-1]`과 비교합니다. 기존 코드에서 함수 `dong_search()`는 `dong_code`를 반환하는 부분이 있었지만, 여기서는 전역변수 `dong_code1`, `dong_code2`에 저장되므로 `return`으로 값을 반환하는 부분은 삭제해도 됩니다. 그리고 행정동명이 행정동코드로 제대로 변환됐다는 것을 사용자가 알 수 있게 `set()` 함수로 `info` 레이블에 값을 보여 줍니다.

> TIP 사실 `lambda`라는 익명함수를 사용하면 버튼 이벤트 연결에서도 인자를 전달할 수 있지만, 이 책에서는 다루지 않았기 때문에 전역변수를 활용합니다.

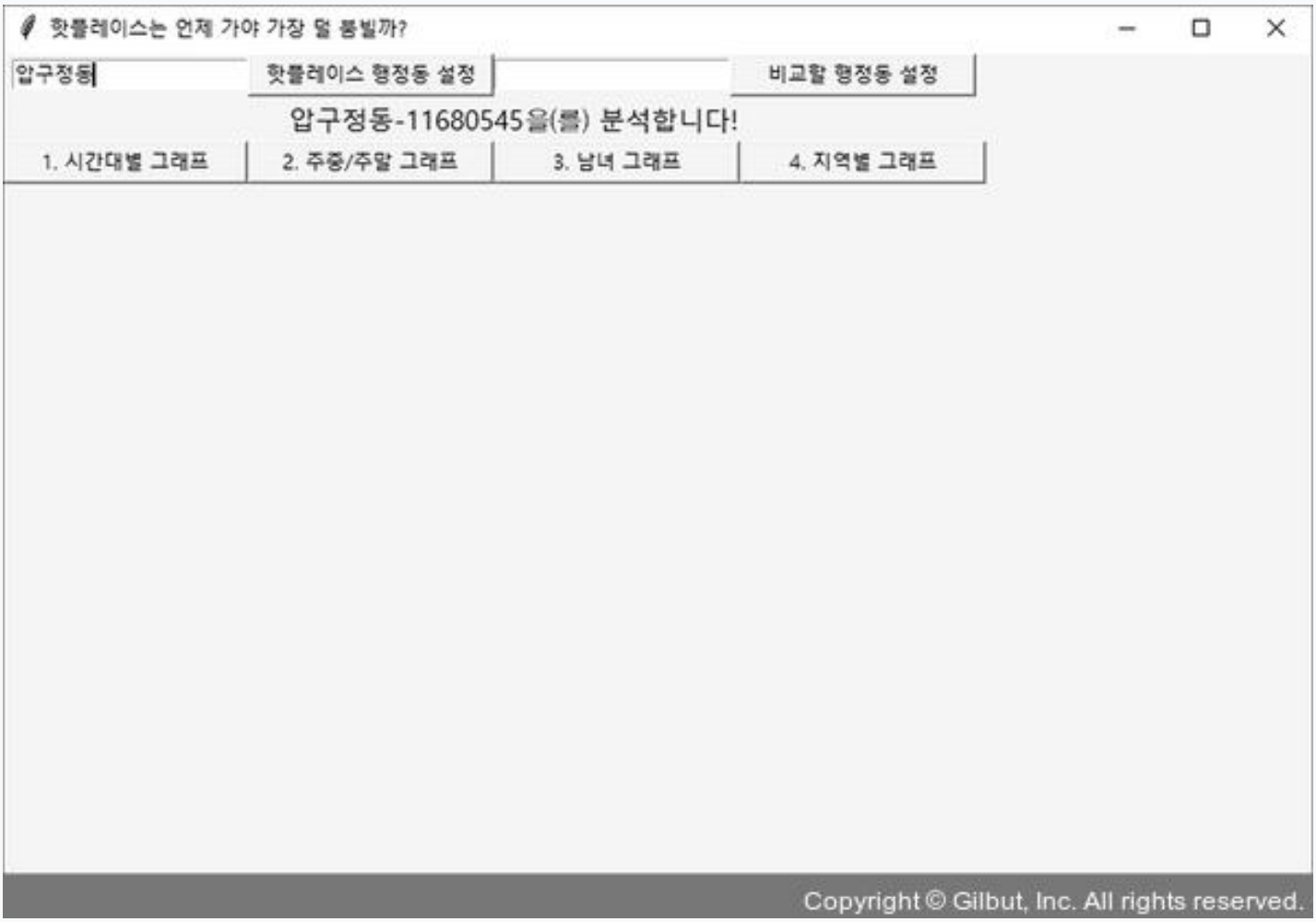
이제 함수를 버튼에 이벤트로 연결할 차례입니다. 프로그램 화면 부분에서 [핫플레이스 행정동 설정] 버튼에는 `dong_search1()` 함수를, [비교할 행정동 설정] 버튼에는 `dong_search2()` 함수를 이벤트로 연결해 봅시다.

```
# ----- 메뉴 프레임 -----
…
dong_btn1 = Button(Menu, text='핫플레이스 행정동 설정', width=20, command=dong_search1)
dong_btn1.grid(row=0, column=1)

…
dong_btn2 = Button(Menu, text='비교할 행정동 설정', width=20, command=dong_search2)
dong_btn2.grid(row=0, column=3)

…
```

실행결과



엔트리 두 곳에 행정동명을 입력해서 이벤트 연결이 잘 되나 확인해 보세요. 행정동명을 입력하고 버튼을 차례대로 클릭하면 `info` 레이블에 행정동명과 코드가 나타납니다. 이렇게 `Menu` 프레임을 완성했습니다.

그래프 그리기

다음으로 `Graph` 프레임 부분을 작성해 봅시다. 먼저 그래프 그리는 `graph_plot()` 함수입니다. `graph_plot()` 함수는 네 가지 분석 버튼을 눌렀을 때 분석 버튼에 이벤트로 연결된 함수들이 공통으로 호출하는 함수입니다. 따라서 그래프로 그릴 인구와 그래프 제목 등을 인자로 입력받아 그래프를 그리고 레이블 `graph_lbl` 위치에 나타내면 됩니다.

화면 위에 `matplotlib`으로 그래프를 직접 그릴 수 있으나 방법이 복잡해 이 책에서는 다루지 않습니다. 그래서 화면에 그래프를 직접 그리는 대신 그래프를 이미지로 저장한 후, 이를 `PhotoImage` 객체로 저장해 `graph_lbl` 레이블에 표시하는 방법을 사용하겠습니다. `dong_search2()` 함수 아래에 `graph_plot()` 함수를 정의하는 코드를 다음과 같이 추가합니다. `graph_plot()`은 버튼에 이벤트로 연결될 함수가 아니기 때문에 매개변수를 사용할 수 있습니다.

```
import matplotlib.pyplot as plt

# 그래프 그리는 함수
def graph_plot(popu_list, label_list, graph_title):
    plt.figure(figsize=(10, 5)) # 그래프 크기 설정
    plt.rc('font', family='Malgun Gothic')
    plt.title(graph_title)
    for i in range(len(popu_list)):
        plt.plot(range(24), popu_list[i], label=label_list[i])
    plt.legend()
    plt.xlabel('시간대')
    plt.ylabel('평균인구수')
    plt.xticks(range(24), range(24))
    plt.savefig('graph.png') # 결과 그래프를 이미지로 저장하기
    plt.show()

    # 그래프 이미지를 PhotoImage의 객체로 만들고, graph_lbl 레이블에 표시하기
    graph_img = PhotoImage(file='graph.png')
    graph_lbl.configure(image=graph_img)
    graph_lbl.image = graph_img
```

그래프를 이미지로 저장하는 명령어는 `plt.savefig('이미지 파일')`입니다(확장자도 있어야 합니다). 실행하면 파이썬 파일이 위치한 폴더에 지정한 이름으로 그래프가 저장됩니다. 이 명령어는 `plt.show()` 바로 앞에 있어야 합니다. 이렇게 `graph_plot()` 함수에서 저장한 그래프 이미지가 `graph_lbl` 레이블에 표시되어야 합니다. 처음에는 비어 있던 `graph_lbl`에 이미지를 나타내는 방법은 다음과 같습니다.

형식

```
이미지명 = PhotoImage(file='이미지 파일')
레이블명.configure(image=이미지명)
레이블명.image = 이미지명
```

먼저 이미지로 저장한 그래프를 `PhotoImage`의 객체로 만듭니다. 그리고 `configure(image=이미지명)`으로 레이블에 이미지를 보여 주게 합니다. 마지막으로 `레이블명.image = 이미지명`으로 이미지를 레이블에 나타냅니다.

TIP `configure()` 메서드에 관한 자세한 내용은 `help(Label.configure)`를 실행해 확인해 보세요.

이 방법에 따라 `graph_img = PhotoImage(file='graph.png')`로 이미지 객체를 만든 후 `graph_lbl.configure(image=graph_img)`와 `graph_lbl.image = graph_img`로 `graph_lbl` 레이블에 이미지 객체 `graph_img`를 보여 주게 설정하면 됩니다.

인구 비교하기

첫 번째 버튼인 [1. 시간대별 그래프] 버튼에 연결할 이벤트인 `analysis1()` 함수를 13장을 참고해서 작성해 봅시다. 버튼에 연결되는 이벤트 함수라서 인자로 데이터를 전달받는 매개변수가 없어야 합니다. 그리고 두 지역을 비교하는 [4. 지역별 그래프] 버튼에서는 `analysis1()`을 호출하기 때문에 매개변수 `graph_yn = 'y'`는 필요합니다(코드는 거의 같으므로 변경된 부분만 표시합니다).

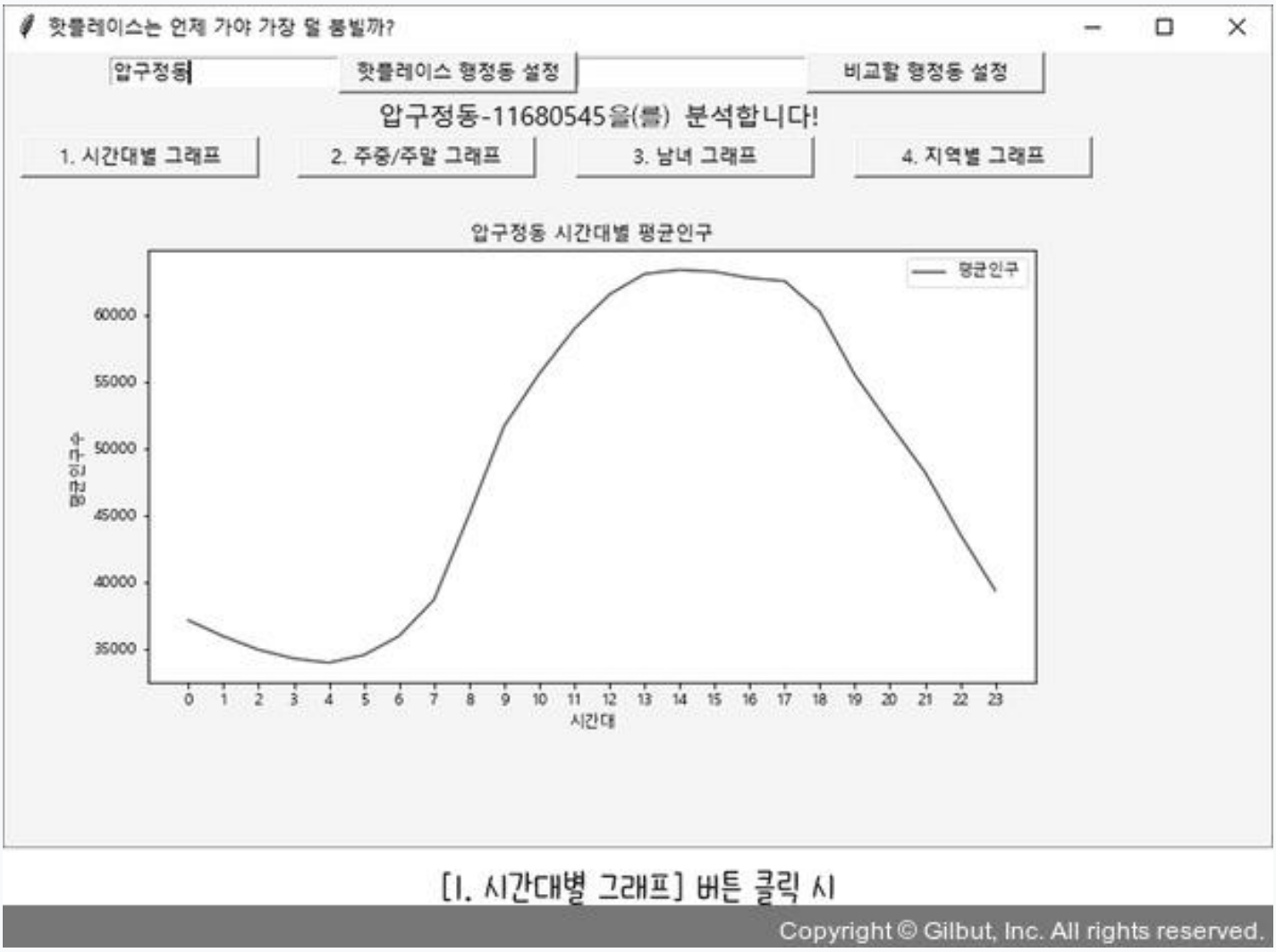
```
def analysis1(graph_yn = 'y'):
    population = [0 for i in range(24)]
    for row in data:
        if row[2] == dong_code1:
            time, p = row[1], row[3]
            population[time] += p
    population = [p/31 for p in population]
    if graph_yn == 'n':
        return population

    population = [population]
    labels = ['평균인구']
    title = dong_name1 + ' 시간대별 평균인구'
    graph_plot(popu_list = population, label_list = labels, graph_title = title)
```

`analysis1()` 함수 작성이 끝나면 [1. 시간대별 그래프] 버튼에 이벤트로 연결합니다. 그리고 프로그램을 실행해 그래프가 나타나는지 확인해 보세요.

```
btn1 = Button(Graph, text='1. 시간대별 그래프', width=20, command=analysis1)
```

실행결과



[1. 시간대별 그래프] 버튼 클릭 시

Copyright © Gilbut, Inc. All rights reserved.

다른 분석도 동일하게 진행하면 됩니다. 데이터를 분석하는 함수는 매개변수를 삭제하고, `dong_name`과 `dong_code`를 각각 `dong_name1`, `dong_code1`로 변경하는 것 외에는 다른 점이 없습니다. `analysis2()`, `analysis3()`도 작성합시다(코드는 거의 같으므로 변경된 부분만 표시합니다).

```
import datetime

def analysis2():
    weekday = [0 for i in range(24)]
    weekend = [0 for i in range(24)]

    for row in data:
        if row[2] == dong_code1:
            time, p = row[1], row[3]
            year, mon, day = int(row[0][:4]), int(row[0][4:6]), int(row[0][6:])
            num = datetime.date(year, mon, day).weekday()
            if num < 5:
                weekday[time] += p
            else:
                weekend[time] += p

    weekday_cnt, weekend_cnt = 0, 0
    for i in range(1, 32):
        if datetime.date(2019, 12, i).weekday() < 5:
            weekday_cnt += 1
        else:
            weekend_cnt += 1
    weekday = [w/weekday_cnt for w in weekday]
    weekend = [w/weekend_cnt for w in weekend]

    data_set = [weekday, weekend]
    labels = ['주중', '주말']
    title = dong_name1 + ' 주중/주말 시간대별 평균인구'
    graph_plot(popu_list = data_set, label_list = labels, graph_title = title)

def analysis3():
    male = [0 for i in range(24)]
    female = [0 for i in range(24)]

    for row in data:
        if row[2] == dong_code1:
            time = int(row[1])
            male[time] += sum(row[4:18])
            female[time] += sum(row[18:32])

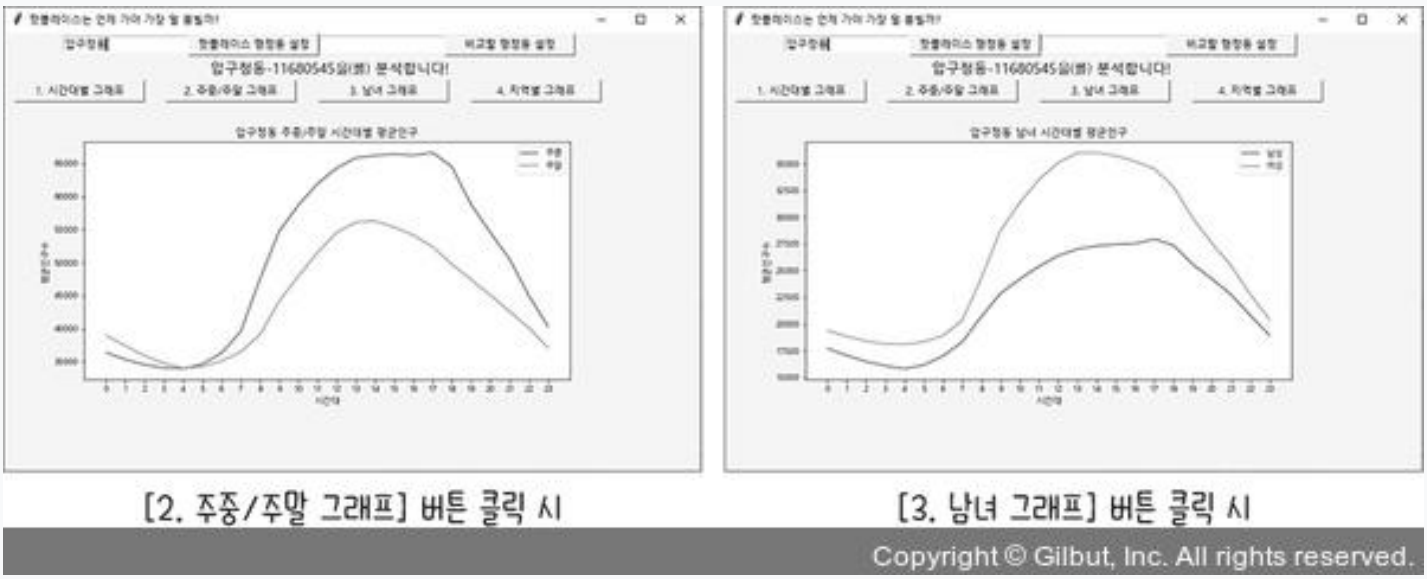
    male = [m/31 for m in male]
    female = [f/31 for f in female]
```

```
data_set = [male, female]
labels = ['남성', '여성']
title = dong_name1 + ' 남녀 시간대별 평균인구'
graph_plot(popu_list = data_set, label_list = labels, graph_title = title)
```

완성된 함수는 버튼에 이벤트로 연결합니다.

```
# ----- 프로그램 화면 부분 -----
…
btn2 = Button(Graph, text='2. 주중/주말 그래프', width=20, command=analysis2)
…
btn3 = Button(Graph, text='3. 남녀 그래프', width=20, command=analysis3)
…
```

실행결과



각각의 버튼을 클릭했을 때 두 그래프가 화면에 잘 나타납니다. 화면에 그래프 이미지가 나타나지 않으면 코드 결과창을 보면서 어떤 부분에서 오류가 발생하는지 추적해 틀린 부분을 수정해 보세요.

마지막으로 [4. 지역별 그래프] 버튼을 클릭할 때 실행되는 `analysis4()` 함수를 작성해 봅시다. 13장에서는 `analysis1()` 함수에 매개변수 `dong_name`, `dong_code`가 있어서 핫플레이스가 있는 행정동명과 코드, 비교할 행정동명과 코드를 인자로 전달하기만 하면 두 행정동의 평균인구를 구해 비교했습니다. 하지만 `tkinter`에서는 매개변수를 사용하지 않으므로 13장의 코드와는 다른 방식으로 접근해야 합니다.

`analysis1()`에서 이미 핫플레이스 행정동의 평균인구를 구했으므로 `analysis4()`에서는 `dong_name2`, `dong_code2`로 비교할 행정동의 평균인구를 직접 구하고 주중/주말 인구나 성별 인구를 비교할 때처럼 비교 그래프를 그리면 됩니다.

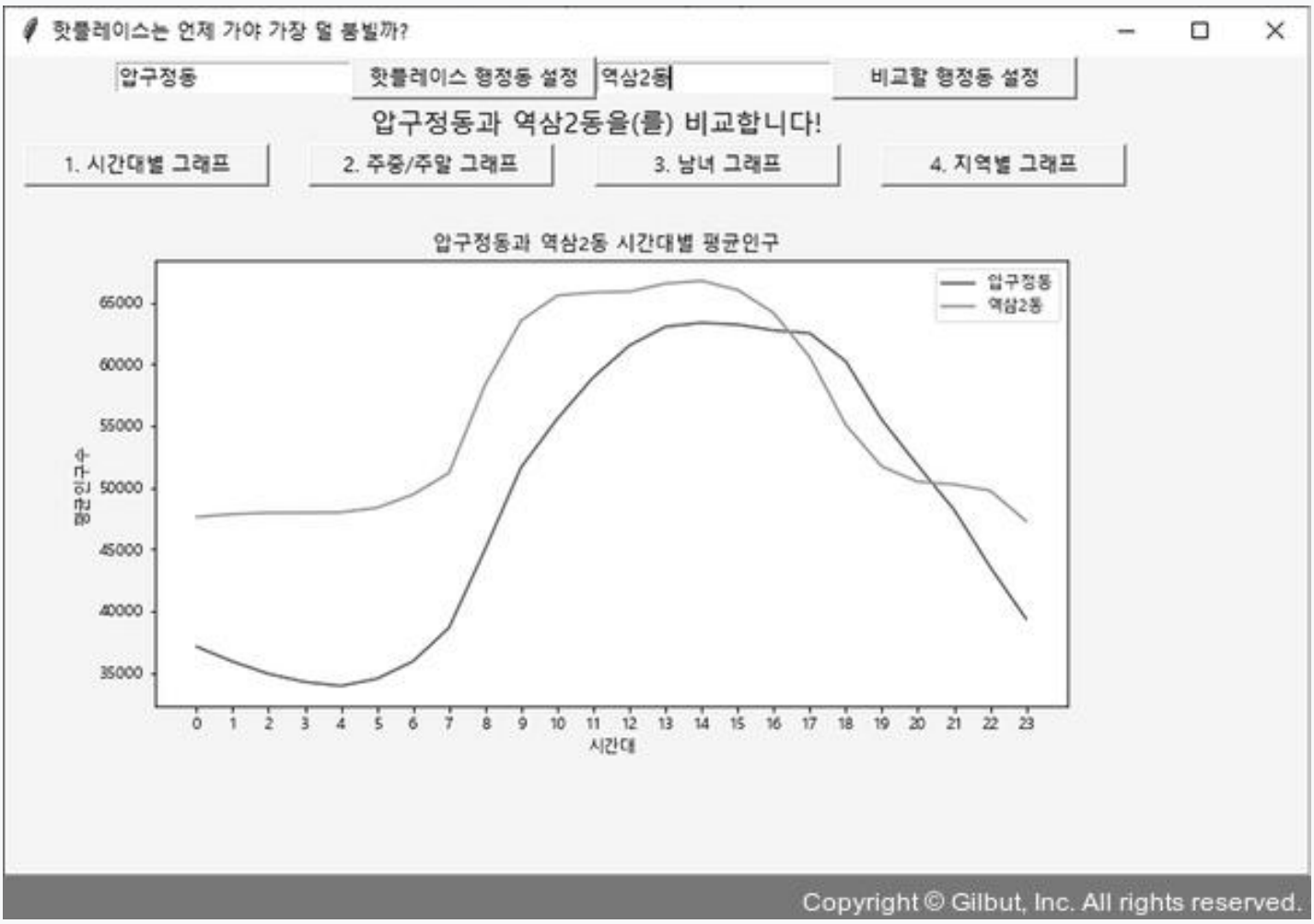
```
def analysis4():
    population1 = analysis1(graph_yn = 'n')
    population2 = [0 for i in range(24)]
    for row in data:
        if row[2] == dong_code2:
            time, p = row[1], row[3]
            population2[time] += p
    population2 = [p/31 for p in population2]

    data_set = [population1, population2]
    labels = [dong_name1, dong_name2]
    title = dong_name1 + '과 ' + dong_name2 + ' 시간대별 평균인구'
    graph_plot(popu_list = data_set, label_list = labels, graph_title = title)
```

작성한 함수를 `btn4` 버튼에 이벤트로 연결한 후 실행해 봅시다.

```
btn4 = Button(Graph, text='4. 지역별 그래프', width=20, command=analysis4)
```

실행결과



두 지역의 인구 그래프가 잘 나타납니다. 핫플레이스 프로젝트에 화면을 입히니 좀 더 프로그램다워지고 인구 분석하기도 좀 더 편해졌습니다. 이를 응용해 다양한 프로젝트를 직접 작성해 보세요! 프로그래밍 실력이 느는 것이 느껴질 겁니다.

전체 프로그램 구조는 다음과 같습니다(전체 코드는 소스 코드에서 확인해 주세요).

```
# ----- 프로그램 기능 부분 -----
import csv
import datetime
import matplotlib.pyplot as plt
from tkinter import *

def file_open() :
    …
def dong_search1() :
    …
def dong_search2() :
    …
def graph_plot(popu_list, label_list, graph_title):
    …
def analysis1(graph_yn = 'y'):
    …
def analysis2():
    …
def analysis3():
    …
def analysis4():
    …

# 전역변수
data, code_data = file_open()     # 파일 읽기
dong_name1, dong_code1 = '', ''   # 핫플레이스가 있는 행정동명과 행정동코드
dong_name2, dong_code2 = '', ''   # 비교할 행정동명과 행정동코드

# ----- 프로그램 화면 부분 -----
root = Tk()
root.geometry('800x500')
root.title('핫플레이스는 언제 가야 가장 덜 붐빌까?')
# ----- 프레임 -----
…
# ----- 메뉴 프레임 -----
…
# ----- 그래프 프레임 -----
…
root.mainloop()
```

파이썬의 기본적인 개념과 사용법을 배우고 이를 적용해 프로젝트까지 구현해 봤습니다. 책에서는 인구 데이터 분석을 진행했지만 직접 데이터를 선정해 나만의 프로젝트를 작성해 보길 권장합니다.

프로그래밍은 아이디어를 현실화할 수 있는 효율적인 도구입니다. 아이디어를 논리적인 순서로 정리하고, 이를 프로그래밍으로 구현해 자동화할 수 있죠. 또한, 프로그래밍은 창의적인 발상을 현실화할 수 있는 훌륭한 도구입니다. 그렇기 때문에 프로그래밍을 전공자만 배워야 하는 것이 아니라, 모든 사람이 자신의 창의성을 실체화할 수 있는 방법으로 프로그래밍을 배워야 한다고 생각합니다.

그래서 이 책은 입문자가 배우기 어렵다고 판단되는 심화적인 부분은 배제하고 핵심 개념을 중심으로 설명했습니다. 예를 들어 클래스와 객체에서 상속 개념을 설명하지 않았고, `matplotlib`을 `fig`, `ax = plt.subplot()` 방식 대신 `plt.plot()`으로 설명한 것 등이 해당됩니다. 하지만 심화 부분을 알지 못하더라도 나만의 프로젝트를 할 수 있습니다. 그래서 이 책은 프로그래밍을 처음 배우는 분도 차근차근 혼자 공부할 수 있게 구성하려 노력했습니다. 만약 이 책을 통해 프로그래밍에 흥미를 느꼈다면 심화 내용을 다룬 다른 도서나 인터넷 검색을 통해 이 흥미를 지속하길 진심으로 기원합니다.

15

마무리

1 tkinter 창 나타내기

```
from tkinter import *

root = Tk()
# ----- 프로그램 화면 부분 -----
root.mainloop()
```

2 tkinter 창 설정하기

구분	형식
창의 제목 설정	`root.title('창의 제목')`
창의 크기 설정	`root.geometry('가로 크기x세로 크기')`
창이 나타나는 위치 설정	`root.geometry('가로 크기x세로 크기+가로 위치+세로 위치')`
창 크기 변경가능 여부 설정	`root.resizable(너비변경여부, 높이변경여부)` 변경할 수 있다면 `True`, 변경할 수 없다면 `False`

3 위젯

1) 위젯 만들기

형식

```
# 텍스트 레이블 객체 생성하기
레이블명 = Label(레이블위치, text='텍스트')
# 이미지 레이블 객체 생성하기
이미지명 = PhotoImage(file='이미지 파일')
레이블명 = Label(레이블위치, image=이미지저장변수)
# 버튼 객체 생성
버튼명 = Button(버튼위치, text='버튼에 나타낼 텍스트')
# 엔트리 객체 생성
엔트리명 = Entry(입력칸위치)
    # 콤보박스 객체 생성
    from tkinter.ttk import *
    콤보박스명 = Combobox(콤보박스위치, values='콤보박스에 나타낼 값(리스트)')
    # 정보제공 메시지박스 띄우기
    from tkinter.messagebox import *
    showinfo(title='메시지 창 제목', message='나타낼 메시지')
    # 오류 메시지박스 띄우기
    from tkinter.messagebox import *
    showerror(title='메시지 창 제목', message='나타낼 메시지')
```

2) 위젯에 공통으로 적용할 수 있는 옵션

구분	형식	옵션 값
위젯 너비 설정	`width=너비`	
위젯 높이 설정	`height=높이`	
위젯 내 텍스트 또는 이미지 정렬	`anchor=정렬방식`	`center`, `e`, `w`, `s`, `n`, `se`, `sw`, `ne`, `nw`
테두리 설정	`relief=테두리모양`	`flat`, `groove`, `raised`, `ridge`, `solid`, `sunken`
테두리 두께 설정	`borderwidth=테두리두께`	

3) 위젯에 추가로 적용할 수 있는 옵션과 명령어

위젯	설명	형식
버튼	이벤트 설정 옵션	`command=함수명`
콤보박스	콤보박스에 보여 줄 값 설정	`콤보박스명.current(숫자)`
	콤보박스에 새로운 값 설정	`콤보박스명['values'] = 리스트`

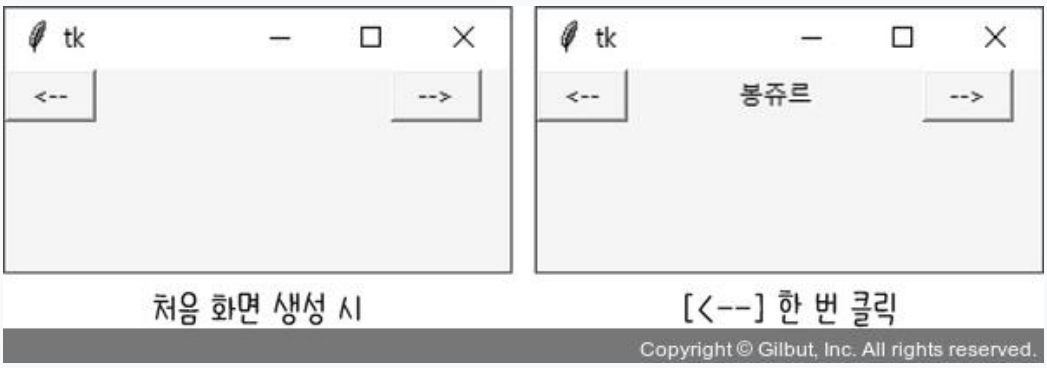
4 위젯 배치하기

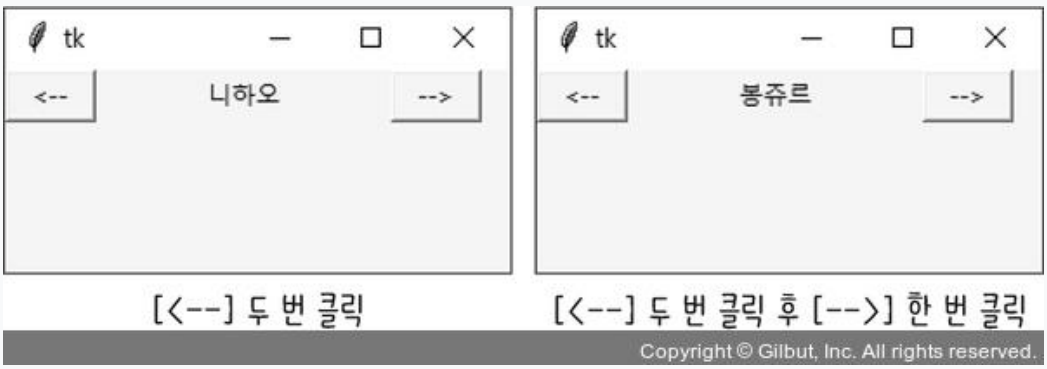
구분	함수	형식	설명
절대위치	`place()`	`위젯명.place(x=x좌표, y=y좌표)`	좌표를 숫자로 직접 입력
상대위치	`pack()`	`위젯명.pack()`	위젯 간 상대위치 사용 위치 설정 옵션 `side`
	`grid()`	`위젯명.grid(row=행, column=열)`	격자를 활용한 상대위치 사용

Self Check

1 1분 퀴즈 2의 코드에 버튼 하나를 추가해 다음 화면처럼 나타나도록 작성해 보세요. [<--] 버튼은 [-->] 버튼과는 반대되는 기능으로, 현재 `sayhello` 리스트의 위치에서 왼쪽에 위치한 값을 화면에 나타냅니다.

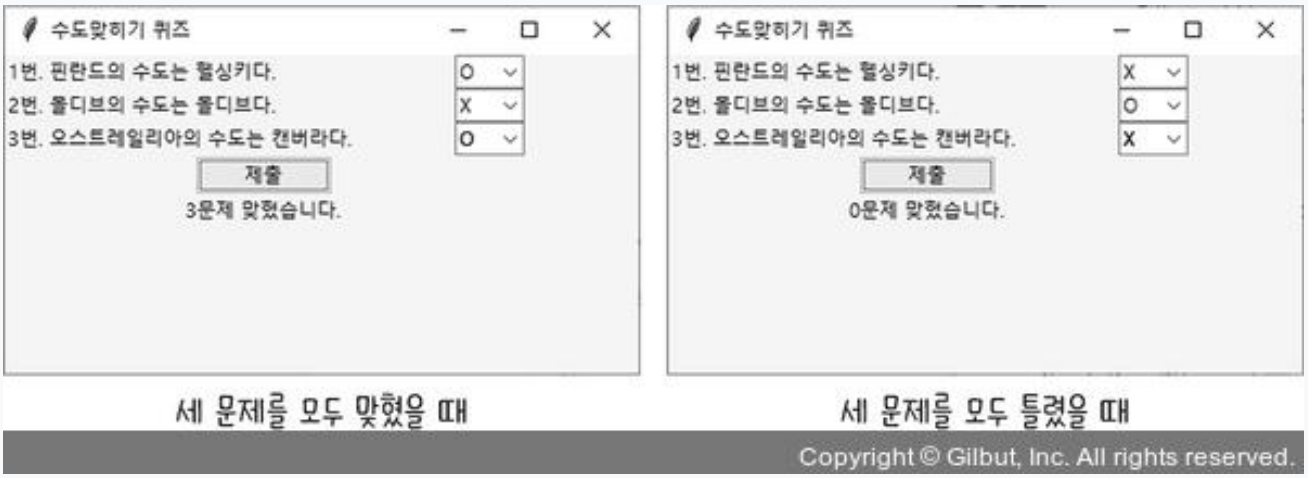
실행결과





2 퀴즈 프로그램을 `tkinter`를 활용해 작성하려고 합니다. 문제 3개를 출제한 후, 맞는지 틀린지 콤보박스로 된 O, X 중 하나를 선택하게 하고 몇 개가 맞았는지 알려 줍니다.

실행결과



다음은 앞의 프로그램을 만드는 코드입니다. `summit`을 정의하고, (가)~(마)에 알맞은 코드를 작성해 프로그램을 완성하세요. 퀴즈 프로그램의 정답은 1번은 O, 2번은 X, 3번은 O입니다.

```
from tkinter import *
from tkinter.ttk import *

def summit():
    # 이 부분을 작성하세요.

root = Tk()
root.title('수도맞히기 퀴즈')
root.geometry('400x200')

result = StringVar()
ox =['O', 'X']

# 1번 문제
lbl1 = Label(root, text='1번. 핀란드의 수도는 헬싱키다.', width=40)
lbl1.grid(row=0, column=0)

combo1 = Combobox(root, (가), width=3)
combo1.grid(row=0, column=1)

# 2번 문제
lbl2 = Label(root, text='2번. 몰디브의 수도는 몰디브다.', width=40)
lbl2.grid(row=1, column=0)

combo2 = Combobox(root, (가), width=3)
combo2.grid(row=1, column=1)

# 3번 문제
lbl3 = Label(root, text='3번. 오스트레일리아의 수도는 캔버라다.', width=40)
lbl3.grid(row=2, column=0)

combo3 = Combobox(root, (가), width=3)
combo3.grid(row=2, column=1)

btn = Button(root, text='제출', (나))
btn.grid((다), (라), columnspan=2)

lbl_result = Label(root, (마), width=35, anchor='center')
lbl_result.grid(row=4, column=0, columnspan=3)

root.mainloop()
```

정답 및 해설